mosaickit 套件：二維圖形組裝工具

Pin Yue Sung[*†]　　Ling Tak Douglas Chung[*]

2026-10-23 · v1.0.0rc6 · ·

目錄

1 簡介 2
1.1 設計 2
1.2 數學與證明 2
1.3 閱讀指引 2
2 安裝 3
2.1 系統需求 3
2.2 安裝套件 3
2.3 開發環境 3
3 快速入門 4
4 畫布與場景 5
4.1 畫布規格 5
4.2 畫布 5
4.3 場景 6
4.4 錯誤與警告 6
5 圖層與座標軸 6
5.1 共同欄位 6
5.2 基本圖層 7
5.3 座標軸 8
6 座標軸註記 8
6.1 標記、註記與大括號 8
6.2 邊欄的欄位 9
6.3 沿軸分散文字 9
6.4 大括號的軌道 10
7 配置幾何 11
7.1 矩形 11
7.2 方向與線段 12
7.3 多邊形 12
7.4 區域的視覺中心 13
7.5 大括號外形 14
8 區域標籤與點標籤 15
8.1 區域標籤 15
8.2 點標籤 15
8.3 微調標籤 16
8.4 硬性限制 17
8.5 引線標註 17
9 樣式與顏色 18
9.1 稀疏樣式 18
9.2 樣式型別 19
9.3 字型 20
9.4 顏色 21
9.5 色盤 21
10 主題與設定 22
10.1 樣式組與主題 22
10.2 角色解析 23
10.3 註冊領域角色 24
10.4 設定 24
10.5 EconViz 套件共用的設定檔 25
11 參數與網格 26
11.1 參數與運算式 26
11.2 網格 28
12 繪製 29
12.1 繪製器 29
12.2 Matplotlib 繪製器 29
12.3 結果與儲存 30
12.4 快取 30
A 證明 30
A.1 排列與軌道 31
A.2 矩形、線段與多邊形 32
A.3 不可及極點 33
A.4 標籤 35
A.5 樣式、主題與綁定 35
更新紀錄 37
指令索引 39
參考文獻 41

[*]國立政治大學國際經營與貿易學系，臺北，臺灣。
[†]通訊作者。電子郵件：contact@econ-viz.org

第 1 節　簡介

mosaickit 套件以小型且不可變的元件組裝二維圖形：場景是一份有序的圖層清單（路徑、填色區域、標記、文字、箭頭、標籤、大括號）；每個圖層標明一個語意角色，主題將角色轉成樣式，繪製器再將結果輸出為 PNG、SVG 或 PDF。它不含任何特定領域的知識。principle-viz、utility-viz 等領域套件自行定義模型與角色名稱，再將要繪製的圖層交給 mosaickit；曲線建構與 TikZ 匯出則由 bezierkit 等幾何套件負責。

1.1　設計

套件設計遵循四項原則。

不可變的值　圖層、場景、樣式、主題與規格都是凍結的 dataclass。`Canvas` 是封裝不可變 `Scene` 的流暢建構器；`snapshot()`、`copy()` 與 `bind()` 不會改動其他物件持有的場景。

稀疏樣式　每個樣式欄位都可以是 `None`，表示繼承。圖層自帶的樣式位於畫布覆寫、設定覆寫、主題與基本預設值之上（詳見第 10 節）。

角色而非顏色　圖層只說明自己是什麼（`"primary"`、`"axes.note"`、`"mypkg.boundary"`），外觀由主題決定，而顏色是畫布繪製時才解析的色盤名稱（詳見第 9 節）。

避讓所有內容的配置　區域標籤、點標籤、大括號與座標軸註記會等其他內容畫完後再配置。配置僅依顯示像素進行幾何計算，讓文字不碰到任何線、標記、區域或其他文字（詳見第 7 節、第 8 節）。

1.2　數學與證明

自動配置以多種計算幾何方法為基礎。方向測試用來判斷點位於有向直線的哪一側，奇偶規則則依射線穿越邊界的次數判斷內外；此外還會計算到多邊形邊界的距離，並以每次優先檢查上界最高候選區域的最佳優先搜尋找出區域最深處。座標軸文字則會排列成互不重疊且位移平方和最小的位置。各章以編號的定義、引理、命題與定理說明每個程序的保證，證明則集中在附錄 A；若只想查閱 API，可以略過。樣式與主題的代數（稀疏合併、角色解析）以及參數綁定也採取相同的處理方式。幾何部分的標準參考書為 de Berg et al. (2008)，定理 2 背後的保序最小平方問題則參見 Barlow et al. (1972)。

1.3　閱讀指引

表 1　章節指引

主題	內容	章節
建構	畫布、規格、場景	第 4 節
	路徑、填色、標記、文字、座標軸	第 5 節
	座標軸標記、註記與大括號	第 6 節
配置	配置幾何	第 7 節
	區域標籤與點標籤	第 8 節

主題	內容	章節
外觀	樣式、顏色、色盤	第 9 節
	主題、角色、設定	第 10 節
輸出	參數與網格	第 11 節
	繪製器、快取、儲存	第 12 節

初次使用請先讀第 3 節、第 4 節與第 5 節。本手冊的圖都是 mosaickit 自己的輸出：每張圖由它所示範的畫布或網格繪製，並以印在此處的尺寸存成 PDF。

繪製本手冊各圖的腳本依主題分類，可一次下載：mosaickit-examples.zip。

第 2 節　安裝

2.1　系統需求

mosaickit 需要 Python 3.10 以上、NumPy 1.24 以上、Matplotlib 3.6 以上（4 以下）與負責排版文字的 glyphkit 1.0.0b3 以上（2 以下）；在 Python 3.10 上還會安裝 tomli 來讀取 TOML。

2.2　安裝套件

```
uv add mosaickit                # the library
uv add "mosaickit==1.0.0rc6"    # the version this manual describes
```

1.0.0rc6 是預覽版。在 1.0 正式版發布之前，直接執行 `uv add mosaickit` 或 `pip install mosaickit` 安裝的是 0.9.2；需以 `uv add --prerelease allow mosaickit` 或 `pip install --pre mosaickit` 才能安裝預覽版。

若使用 pip，請執行 `python -m pip install mosaickit`。匯入 `mosaickit` 時不會一併匯入 Matplotlib：內建繪製器要到畫布第一次繪製時，才會依名稱載入（詳見第 12 節）。

2.3　開發環境

```
git clone https://github.com/EconViz/mosaickit.git
cd mosaickit
uv sync --locked
uv run pre-commit install
uv run pytest
uv run ruff check .
uv run ruff format --check .
uv run mypy
uv run lint-imports
uv build
```

鎖定檔固定了所有開發相依套件的版本。持續整合流程會在 Python 3.10、3.11、3.12 與 3.13 上執行相同命令，再將建置完成的 wheel 安裝到乾淨環境，確認其中不含 bezierkit，

最後以該 wheel 執行測試。匯入規則維持分層：場景、樣式、主題與參數模組不會匯入繪製與畫布模組，核心也不會匯入任何領域套件。

第 3 節　快速入門

一張圖由畫布、一組圖層與一次儲存構成：

例 1

```
from mosaickit import (
    Canvas, Fill, FillLayer, MarkerLayer, PathLayer, Stroke, TextLayer,
    quadrant_axes,
)

canvas = Canvas().extend(quadrant_axes(10, 10))
canvas.add(
    FillLayer(
        [(1, 1), (1, 7), (8, 1)],
        fill=Fill(color="#377EB8", opacity=0.12),
        z_index=-1,
    )
)
canvas.add(
    PathLayer(
        [(1, 8), (2, 5), (4, 3), (7, 1.5), (9, 1)],
        stroke=Stroke(color="#984EA3", width=2),
        id="curve",
    )
)
canvas.add(MarkerLayer([(4, 3)], id="point"))
canvas.add(TextLayer((4, 3), "A", offset=(6, 6)))
canvas.save("diagram.pdf")
```

`quadrant_axes(10, 10)` 回傳一般圖層：兩條帶箭頭的路徑及其標題。填色圖層以 `z_index=-1` 放在最底層。路徑是通過各點的折線；mosaickit 本身不直接繪製曲線。若要呈現平滑曲線，必須傳入大量取樣點，或先用幾何套件建構曲線再取樣。文字從該點向右上偏移 6 pt。呼叫 `save()` 之前不會進行任何繪製；`save()` 會用畫布的繪製器（未另行設定時為 Matplotlib）繪製場景、寫出檔案，並回傳輸出路徑。

同一個畫布可在修改後再次儲存：`add()`、`extend()`、`remove()` 與 `clear()` 都會回傳畫布本身，因此可以串接呼叫；先前取得的快照則維持原狀（詳見第 4 節）。能自動避讓其他內容的標籤也是圖層：將 `TextLayer` 換成 `PointLabelLayer((4, 3), "A")`，繪製器就會自行選擇擺放方向（詳見第 8 節）。

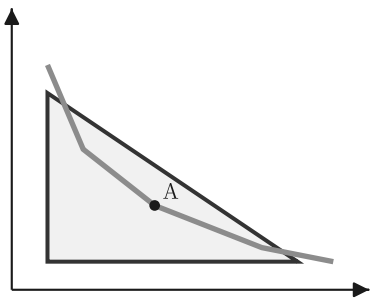


圖 1
快速入門的圖

第 4 節　畫布與場景

4.1　畫布規格

`CanvasSpec` `CanvasSpec(x_range=(0, 10), y_range=(0, 10), width=6.0, height=6.0, dpi=300, x_label="X", y_label="Y", title=None)`
圖形的實體尺寸與座標範圍。`x_range` 與 `y_range` 是顯示的資料範圍；`width` 與 `height` 是以英寸計算的圖形尺寸；`dpi` 是 $[1, 1200]$ 內的整數。屬性 `x_min`、`x_max`、`y_min`、`y_max` 可讀取範圍，`replace(**changes)` 則回傳只修改指定欄位的副本。範圍必須有限且 `lo < hi`，尺寸必須是有限正數，否則會引發 `ConfigurationError`。

`Interval` `Interval(`⟨*lo*⟩`, `⟨*hi*⟩`)`
有限的遞增區間，`lo < hi`，供規格與座標軸共用。

4.2　畫布

`Canvas` `Canvas(spec=None, theme=None, config=None, renderer=None, *, role_overrides=None)`
封裝不可變場景的流暢建構器。若引數為 `None`，便採用建立畫布當下生效的設定（詳見第 10.4 節），分別是其中的 `canvas_spec`、`theme` 與 `renderer`。`role_overrides` 將角色名稱對應至 `StyleBundle`，並套用在主題與設定之上（參見定理 30）。

`add(layer)`, `extend(layers)`, `remove(layer_id)`, `clear()`, `snapshot()`, `copy()`, `bind(parameter, value)`, `bind(mapping)`, `render(*, renderer=None, cache=None)`, `save(target, *, renderer=None, cache=None, **options)`
加入一個或多個圖層；回傳畫布。移除該 id 的圖層（含群組內）或所有圖層；回傳畫布。未知的 id 引發 `ConfigurationError`。目前的 `Scene`。之後呼叫 `add()` 不會改變它。規格、主題、設定、繪製器、覆寫、場景與綁定都相同的新畫布。新畫布，其場景中的這些參數換成數值（詳見第 11 節）；原畫布保留參數。繪製場景並回傳繪製器的結果（詳見第 12 節）。繪製、寫出 `.png`、`.pdf` 或 `.svg`、關閉結果並回傳寫出的路徑。選項即 `SaveOptions` 的欄位。

建構方法會修改畫布，但不會就地改動場景：每次呼叫都以新場景取代畫布中的場景，因此先前取得的快照、副本或綁定後的畫布都會維持原有內容。

```
from mosaickit import Canvas, PathLayer

canvas = Canvas()
before = canvas.snapshot()
canvas.add(PathLayer([(0, 0), (1, 1)], id="line"))
assert before.layers == ()              # the old snapshot is unchanged
assert canvas.snapshot().layers[0].id == "line"
```

4.3　場景

Scene `Scene(layers=(), metadata={})`
具持久性且有序的圖層集合，也就是每次操作都會保留舊版本。`add()`、`extend()`、`remove()` 與 `clear()` 都會回傳新場景；`Scene.empty()` 是空場景。圖層 id 在整個場景（包括群組）中必須唯一，若有重複便引發 `ConfigurationError`。`ordered_layers` 依 `z_index` 排序頂層圖層；值相同時維持加入順序，而這個順序就是繪製順序。

4.4　錯誤與警告

表 2
例外與警告

類別	引發或發出的時機
`MosaicKitError`	以下三個錯誤的基礎類別
`ConfigurationError`	模型、樣式、主題、規格或設定值無效
`BindingError`	參數缺少值、型別錯誤，或運算式無法求值
`RenderError`	繪製器無法完成要求：未知的格式或繪製器、缺少的圖例項目
`LayoutWarning`	自動配置無法避開所有障礙物（詳見第 8 節）
`CacheBypassWarning`	圖層的模型無法雜湊，因此不經快取繪製

第 5 節　圖層與座標軸

5.1　共同欄位

Layer `Layer(*, id=⟨uuid⟩, role="primary", z_index=0, visible=True, legend=None, model=None)`
所有圖層共用的不可變基礎類別。各種圖層在自身的位置引數之後，都接受以下僅限關鍵字使用的欄位。

id 非空字串，在場景中唯一；預設為隨機十六進位字串。圖例、區域標籤與 `remove()` 都透過 id 指定圖層。

role 以點分隔的角色名稱，例如 `"primary"` 或 `"mypkg.boundary"`；它決定圖層從主題取得的樣式（詳見第 10 節）。每種圖層都有自己的預設角色，以及最後才會查詢的後備類別。

z_index 有限的繪製順序；數值越大越晚畫、越在上層。群組會把自己的 `z_index` 加到子圖層上。

visible `False` 時繪製會略過該圖層。

legend 圖層在圖例中的標籤；沒有標籤的圖層不列入圖例。

model 用來建立該圖層的任意領域物件。它會參與繪製快取（詳見第 12 節），但不參與相等比較與綁定。

座標可以是有限實數對（資料單位），也可以是參數運算式（詳見第 11 節）。無效的幾何會在建立圖層時引發 `ConfigurationError`，而不是等到繪製時才報錯。

5.2　基本圖層

表 3　基本圖層

圖層	預設角色	繪製內容
`PathLayer(path, stroke)`	`primary`	通過至少兩點的折線
`FillLayer(boundary, fill, stroke)`	`region`	至少三點的多邊形，含填色與外框
`MarkerLayer(points, marker)`	`point`	每個點上的標記，至少一點
`TextLayer(position, text)`	`text`	某點上的文字
`ArrowLayer(start, end, stroke)`	`annotation`	直線箭頭，可加標籤
`LegendLayer(entries, style)`	`legend`	有標籤圖層的圖例
`GroupLayer(children)`	`primary`	本身不畫任何東西，只將圖層分組

`PathLayer` `PathLayer(path, stroke=None, arrow_placement=ArrowPlacement.END, clip=True, *, ...)`
以直線段連接各點。若解析後的線條樣式設有 `arrow`，便會在 `START`、`END` 或 `BOTH` 端畫出與該端線段對齊的箭頭。`clip=False` 可讓線條以完整寬度越過繪圖區邊緣，座標軸預設組便採用此設定。

`FillLayer` `FillLayer(boundary, fill=None, stroke=None, *, ...)`
以 `fill` 填色、以 `stroke` 描邊的封閉多邊形。`RegionLabelLayer` 以它的 id 指稱它。

`MarkerLayer` `MarkerLayer(points, marker=None, *, ...)`
在每個點繪製一個標記。標記一律完整畫出：中心位於繪圖區邊緣的標記（例如座標軸上的點）不會被切半，中心落在繪圖區外的則會略過。

`TextLayer` `TextLayer(position, text, style=None, offset=(0, 0), anchor="center", math=False, *, ...)`
在 `position` 放置文字，並偏移 `offset` pt。`anchor` 指定文字框的哪一點要對準該位置：`center`、`left`、`right`、`top`、`bottom`、`top-left`、`top-right`、`bottom-left` 或 `bottom-right`。`math=True` 時，文字會以數學式排版（`"a_1"` 成為 a_1）。

`ArrowLayer` `ArrowLayer(start, end, stroke=None, label=None, arrow_placement=ArrowPlacement.END, *, ...)`
從 `start` 指向 `end` 的直線箭頭。除非線條樣式指定其他 `ArrowStyle`，否則使用開放式箭頭；`label` 寫在中點。虛線、點線或點劃線只套用在箭身，箭頭維持實線。箭頭不受繪圖區裁切。

`LegendLayer` `LegendLayer(entries=(), style=None, *, ...)`
列出 `entries` 中各 id 所指定的圖層；`entries` 為空時，則列出所有帶有 `legend` 標籤的圖層。若 id 不存在或對應圖層沒有標籤，便會引發 `RenderError`。`LegendStyle(visible=False)` 可隱藏圖例。

GroupLayer `GroupLayer(children, *, ...)`
由其他圖層組成的圖層。子圖層的繪製方式如同直接加入場景，但其 `z_index` 會再加上群組的值；隱藏群組也會隱藏其中所有子圖層。

5.3　座標軸

AxisSpec `AxisSpec(extent, arrow=None, label=None)`

build_axes `build_axes(x, y) -> list[Layer]`
`build_axes` 將兩個座標軸規格轉成角色為 `axes` 的一般圖層。兩個規格都沒有 `arrow` 時，結果是圍住範圍的封閉框；否則，各軸會沿 $y = 0$ 或 $x = 0$ 畫成直線，並依指定的 `Arrow Placement` 畫出實心三角箭頭。標題位於箭頭尖端之外 6 pt：x 軸標題在右側，y 軸標題在上方。

quadrant_axes `quadrant_axes(x_max, y_max)`

crosshair_axes `crosshair_axes(x_range, y_range)`

box_frame `box_frame(total_x, total_y)`
三種預設組依序為：從原點出發、遠端帶箭頭的兩軸；穿過原點、兩端帶箭頭的兩軸；以及不帶箭頭的外框。

每條座標軸都由 `PathLayer` 與 `TextLayer` 組成，因此可透過 `axes` 角色改變樣式、以 id（`axes.x`、`axes.y`、`axes.frame`、`axes.x.label`、`axes.y.label`）移除，或改用自行建立的圖層。

第 6 節　座標軸註記

y 軸位於繪圖區左緣，x 軸位於下緣，兩軸外側的空間稱為*邊欄*。其中三種圖層配置在邊欄中，第四種則在繪圖區內畫出大括號。

6.1　標記、註記與大括號

AxisMarkLayer `AxisMarkLayer(axis, value, label, math=False, style=None, *, role="axes", ...)`
新增：*0.3.0* 在軸 `"x"` 或 `"y"` 的 `value` 處、繪圖區外的短符號，例如 a_0 或 p^*。

AxisNoteLayer `AxisNoteLayer(axis, value, text, style=None, *, role="axes.note", ...)`
新增：*0.3.0* 用來說明 `value` 的文字，可以分成多行，位於邊欄最外側的一欄。在預設主題中，`axes.note` 角色會讓註記比標記更小、更淡（9 pt、`grey-600`）。

BraceLayer `BraceLayer(axis, start, end, label=None, side="inside", *, role="axes", math=False, style=None, stroke=None, ...)`
新增：*0.3.0* 覆蓋軸上 `start..end` 的大括號，可加上標籤。`side="outside"` 會畫在邊欄中、標記之外；`"inside"` 則畫在繪圖區內側，標籤會避開所有線、點、區域與文字，尖端外沒有空位時便改用引線拉出。

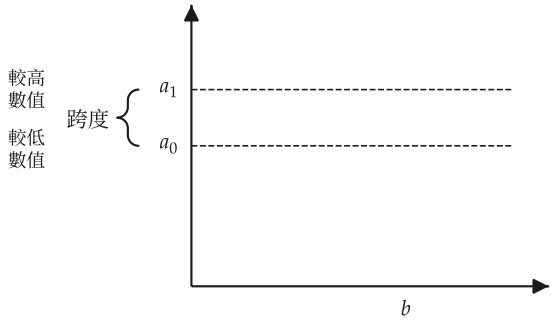


圖 2
y 軸邊欄的標記與註記

`SpanBraceLayer` `SpanBraceLayer(start, end, label=None, side="below", *, role="axes", math=False, style=None, stroke=None, ...)`

新增：*0.4.0*

繪圖區內兩點之間的大括號。跨距必須是水平（`side` 為 `"above"` 或 `"below"`）或垂直（`"left"` 或 `"right"`）；大括號會朝 `side` 凸出，與兩點相距 4 pt、深 8 pt。標籤位於尖端之外，並會避開其他所有內容（詳見第 8 節）。

```
from mosaickit import AxisMarkLayer, AxisNoteLayer, BraceLayer, Canvas,
quadrant_axes

canvas = Canvas().extend(quadrant_axes(10, 10))
for value, symbol, note in [(7, "a_1", "Upper\nvalue"), (5, "a_0",
"Lower\nvalue")]:
    canvas.add(AxisMarkLayer("y", value, symbol, math=True))
    canvas.add(AxisNoteLayer("y", value, note))
canvas.add(BraceLayer("y", 5, 7, "Span", side="outside"))
canvas.add(AxisMarkLayer("x", 6, "b", math=True))
```

6.2　邊欄的欄位

各欄從軸線向外排列，第一欄與軸線相距 5 pt，各欄彼此相距 8 pt。順序依次是標記、每條外側大括號軌道各一欄，以及註記。每欄寬度取其中最寬的內容；空欄不占空間，也不增加間距（參見圖 2，圖中另加了輔助線）。接著還要處理兩個問題：同一欄中相鄰的文字不能重疊，以及哪些大括號可以共用一條軌道。

6.3　沿軸分散文字

同一欄中的每段文字都對應到軸上的一個區間，並以其標示的值為中心。區間重疊時，系統會在維持順序的前提下將它們分開，並讓各項位移的平方和最小。

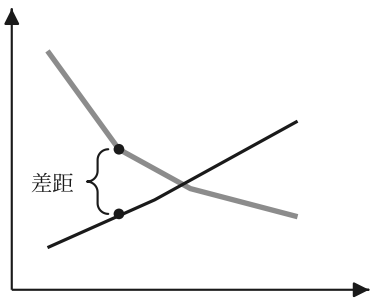


圖 3
兩點之間的跨距大括號

定義 1 (保序排列) 設 $c_1, \dots, c_n$ 為中心、$s_1, \dots, s_n \ge 0$ 為大小、$g \ge 0$ 為間距，編號使 $c_1 \le \cdots \le c_n$（相等者依輸入順序）。*排列*是滿足

$$x_{k+1} - x_k \ge \frac{s_k + s_{k+1}}{2} + g, \quad k = 1, \dots, n-1$$

的向量 $x \in \mathbb{R}^n$；*保序排列問題*是在所有排列中最小化 $\sum_k (x_k - c_k)^2$。

spread `mosaickit.layout.stack1d.spread(centers, sizes, gap=0.0) -> tuple[float, ...]`

新增：*0.3.0* 求解此問題：將連續且重疊的項目合併成群，每群緊密排列在所有成員目標位置的平均值周圍；只要某群碰到前一群，就再次合併。結果依輸入順序回傳。長度不一致或大小為負時會引發 `ValueError`。

定理 2 (分散為最佳解) *spread 回傳定義 1 保序排列問題的唯一解。*

這個程序採用保序迴歸的相鄰違反者合併演算法（pool-adjacent-violators algorithm，PAVA）(Ayer et al., 1955)：扣除緊密排列的位移後，間距限制就會變成 $y_1 \le \cdots \le y_n$。

推論 3 (分散結果的性質) *設 x 為 spread 的結果，則 (i) 任兩項 $i \ne j$ 滿足 $|x_i - x_j| \ge \frac{s_i + s_j}{2} + g$；(ii) 若各中心本身已構成排列，則 $x = c$；(iii) 每群的位移總和為零，因此群的平均位置等於其成員的平均目標。*

圖 4 中的前三項因重疊而形成一群，群的位置以各項目標位置的平均值為中心；其餘兩項原本就已分開，因此不會移動。

6.4　大括號的軌道

跨距（連同標籤）彼此太近的大括號必須放在不同軌道，也就是與軸線不同的距離。

定義 4 (衝突區間) 對間距 $g \ge 0$，若 $b + g \le a'$ 與 $b' + g \le a$ 都不成立，則區間 $[a, b]$ 與 $[a', b']$ *衝突*。*軌道指派*為每個區間指定一個軌道編號，使衝突的區間不共用軌道。

assign_lanes `mosaickit.layout.stack1d.assign_lanes(intervals, gap=0.0) -> tuple[int, ...]`

新增：*0.3.0* 首次適配：依輸入順序處理區間，把每個區間放進與已有區間都不衝突的最低軌道。端點可依任一順序給出。



圖 4
目標與分散結果

定理 5 (首次適配使用最少軌道) *assign_lanes 一定回傳軌道指派。若區間依下端遞增的順序給出，且每個都滿足 $b - a + g > 0$，則它恰好使用 ω 條軌道，其中 ω 為兩兩衝突的區間數的最大值；沒有任何軌道指派能用得更少。*

若輸入不是上述順序，首次適配可能使用超過必要數量的軌道。因此，可能重疊的大括號應依數值由低到高加入。

gutter_columns　`mosaickit.layout.gutter.gutter_columns(mark_width, brace_widths, note_width, *, start, gap)`

新增：*0.3.0*　負責邊欄欄位配置的純函式：回傳 `GutterColumns`，其中包含 `marks`、`braces`（每條軌道一個 `Band`，由內而外）與 `notes`。每個欄位都是從軸線向外量得的 `Band(near, far)`，另有最遠邊緣 `extent`。

第 7 節　配置幾何

標籤會等其他內容全都畫完後再配置，並採用以像素為單位、y 向上遞增的顯示座標。`mosaickit.layout` 套件收錄配置所需的幾何功能；它不匯入任何繪製器，因此可單獨測試及重複使用。本章的函式位於 `mosaickit.layout.geometry`；`Rect` 與 `polylabel` 也會從 `mosaickit.layout` 匯出。點是數對 `(x, y)`，線段由一對點組成，多邊形則是一串首尾相連的點。

7.1　矩形

Rect　`Rect(x0, y0, x1, y1)`

閉合的軸對齊矩形 $[x_0, x_1] \times [y_0, y_1]$。`Rect.centered(center, width, height)` 以指定點為中心建立矩形。它提供 `width`、`height`、`center`、`inflate(pad)`、`corners()`（從 (x_0, y_0) 開始逆時針排列）、`edges()`、`contains(point)`（包含邊界）、`within(other)`、`intersects(other)`（相切也算）與 `nearest_point(point)`。

引理 6 (矩形上的最近點) *對矩形 R 與點 p，nearest_point 回傳的點*

$$q = \Big(\min(\max(p_x, x_0), x_1), \min\Big(\max\big(p_y, y_0\big), y_1\Big)\Big)$$

是 R 上離 p 最近的唯一點。

引線標註的引線會終止於這一點，因此標籤位置一旦確定，引線便是最短的。

7.2　方向與線段

定義 7 (方向) 對平面上的點 a, b, c，

$$\operatorname{orient}(a,b,c) = (b_x - a_x)\big(c_y - a_y\big) - \big(b_y - a_y\big)(c_x - a_x),$$

即 $b-a$ 與 $c-a$ 的外積。

引理 8 (方向測試) *當 c 位於從 a 指向 b 的有向直線左側（$a \to b \to c$ 為逆時針轉向）時，$\operatorname{orient}(a,b,c)$ 為正；位於右側時為負；a、b、c 共線時為零。它在循環置換下不變，交換兩個引數時變號。*

`segments_intersect` `segments_intersect(a, b, c, d) -> bool`
判斷閉線段 $[a,b]$ 與 $[c,d]$ 是否有共同點。令 $d_1 = \operatorname{orient}(c,d,a)$、$d_2 = \operatorname{orient}(c,d,b)$、$d_3 = \operatorname{orient}(a,b,c)$、$d_4 = \operatorname{orient}(a,b,d)$；若 $d_1 d_2 < 0$ 且 $d_3 d_4 < 0$，或某個 d_i 為零且其對應點落在另一線段的外接矩形（包住該線段的最小軸對齊矩形）內，便判定兩者相交 (Cormen et al., 2009)。

定理 9 (線段相交) *在精確算術下，`segments_intersect(a, b, c, d)` 為真若且唯若 $[a,b] \cap [c,d] \neq \emptyset$。*

`rect_hits_segment` `rect_hits_segment(rect, a, b) -> bool`
線段是否碰到矩形：有端點在矩形內，或線段與矩形四邊之一相交。

引理 10 (線段與矩形) *`rect_hits_segment(R, a, b)` 為真若且唯若 $[a,b] \cap R \neq \emptyset$。*

7.3　多邊形

定義 11 (簡單多邊形) 若多邊形 $P = (p_0, \dots, p_{m-1})$（$m \geq 3$）的各邊 $[p_i, p_{i+1}]$（指標取模 m）只會在相鄰邊的共同端點相交，且所有頂點不全共線，便稱 P *為簡單多邊形*。其邊界 ∂P 是所有邊的聯集。依 Jordan 曲線定理，∂P 的補集恰有一個有界連通分量，也就是*內部* $\operatorname{int} P$，以及一個無界分量，也就是*外部*。記 $\overline{P} = \operatorname{int} P \cup \partial P$。

`polygon_edges` `polygon_edges(polygon) -> tuple[Segment, ...]`

`distance_to_boundary` `distance_to_boundary(point, polygon) -> float`
前者回傳多邊形的所有邊（包括最後一點到第一點），後者計算指定點到最近邊的歐氏距離。計算每條邊的距離時，會先將點投影到該邊所在的直線，再把參數限制在 $[0,1]$。

引理 12 (到線段的距離) 設 $a \neq b$，令

$$t^* = \operatorname{clamp}\big((p-a)\cdot(b-a)/\,|b-a|^2, 0, 1\big)$$

則 $a + t^*(b-a)$ 是 $[a,b]$ 上離 p 最近的點。

`point_in_polygon` `point_in_polygon(point, polygon) -> bool`
奇偶規則：從該點向右作水平射線，計算它穿過的邊數。當一條邊恰有一個端點嚴格位於射線所在直線上方，且交點在該點右側時，該邊計入 (Haines, 1994)。

命題 13 (奇偶規則) 設 P 為簡單多邊形且 $p \notin \partial P$。則 *`point_in_polygon(p, P)` 為真若且唯若* $p \in \operatorname{int} P$。

`rect_inside_polygon` `rect_inside_polygon(rect, polygon) -> bool`

`rect_overlaps_polygon` `rect_overlaps_polygon(rect, polygon) -> bool`
判定在內部時，四個角都必須通過奇偶測試，而且多邊形的任何一條邊都不能碰到矩形。判定重疊時，只要有一個角通過測試，或有一條邊碰到矩形即可（這也涵蓋多邊形完全位於矩形內的情形）。

定理 14 (多邊形內的矩形) 設 P 為簡單多邊形、R 為矩形。則 *`rect_inside_polygon(R, P)` 為真若且唯若* $R \subset \operatorname{int} P$。

推論 15 (與多邊形重疊的矩形) 設 P 為簡單多邊形、R 為矩形。則 *`rect_overlaps_polygon(R, P)` 為真若且唯若* $R \cap \overline{P} \neq \emptyset$。

`ray_exit` `ray_exit(origin, direction, polygon) -> float`
回傳射線 $o + tr$ 與不平行於它的邊相交時，所有 $t \geq 0$ 中的最大值；若沒有這樣的邊，則回傳 0。引線標註會將這個位置，也就是射線離開區域之處，當成搜尋起點。

命題 16 (永久離開多邊形) 設 P 為簡單多邊形、$r \neq 0$，且 $T =$ *`ray_exit(o, r, P)`*。則對每個 $t > T$，$o + tr \notin \overline{P}$。

7.4　區域的視覺中心

區域的形心，也就是將區域視為均勻薄片時的質心，可能落在區域外（參見圖 5）。標籤應放在區域最深處，也就是離邊界最遠的點。

定義 17 (有號距離與不可及極點) 對簡單多邊形 P，有號距離定義為：當 $p \in \overline{P}$ 時 $f(p) = d(p, \partial P)$，否則 $f(p) = -d(p, \partial P)$。P 的不可及極點是 f 取得最大值 f^* 之處；f^* 即 P 內最大圓盤的半徑。

引理 18 (有號距離為 1-Lipschitz) 對所有點 p, q，$|f(p) - f(q)| \leq |p - q|$。

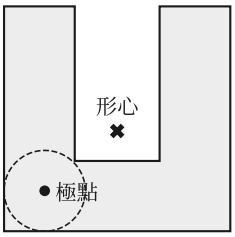


圖 5
極點最大圓盤與形心

`polylabel` `polylabel(polygon, precision=1.0) -> Point`

在正方形格子上執行最佳優先搜尋 (Agafonkin, 2016)。首先用邊長等於外接矩形短邊的正方形覆蓋該矩形。中心為 c、半邊長為 h 的格子，其上界為 $f(c) + h\sqrt{2}$；格子會放入優先佇列，並優先取出上界最大者。程序會記錄目前最佳的中心，初始值為外接矩形的中心。若取出格子的上界比目前最佳值高出 `precision` 以上，便將它分成四格，否則捨棄。若多邊形的外接矩形寬或高為零，則回傳其第一個頂點。

定理 19 (不可及極點) *設 P 為簡單多邊形，外接矩形的寬與高皆為正，並設精度 $\varepsilon > 0$。則 `polylabel` 必定結束，且回傳的點 q 滿足 $f(q) \geq f^* - \varepsilon$。特別地，當 $f^* > \varepsilon$ 時，q 位於* $\operatorname{int} P$。

三角形的極點就是內心，也就是三條角平分線的交點。配置時會以預設的一像素精度呼叫 `polylabel`，這比任何文字的擺放精度都更細。

7.5　大括號外形

`brace_outline` `brace_outline(start, end, *, base, depth, direction, axis="y", samples=12)`

`Brace`

新增：*0.3.0*

建立覆蓋 `start..end` 的大括號，以 `Brace(points, tip)` 表示。兩端位於直線 `base`（固定的橫向座標），尖端位於跨距中點，並在 `direction`（± 1）一側距 `base` 為 `depth`。軸為 `"y"` 時，各點表示為（橫向, 縱向）；軸為 `"x"` 時，則表示為（縱向, 橫向）。每段四分之一圓弧會以 `samples` 段取樣。若跨距為空、深度不是正數，或方向與軸使用其他值，便會引發 `ValueError`。

外形會先在局部座標中建構，其中 u 是橫向，v 是沿軸方向。四段半徑為 $r = \min\left(\frac{\text{depth}}{2}, \frac{\text{end} - \text{start}}{4}\right)$ 的四分之一圓弧以兩段直線相連，再沿橫向拉伸 $\text{depth}/2r$ 倍。因此，跨距較短時圓弧也會較小，但尖端仍能達到完整深度（參見圖 6）。

命題 20 (大括號的形狀) *設 $\ell < h$ 為跨距兩端、$m = (\ell + h)/2$ 為中點、δ 為深度。在局部座標中、取樣之前，外形是從 $(0, \ell)$ 經尖端 (δ, m) 到 $(0, h)$ 的曲線，且 (i) 位於帶狀區 $0 \leq u \leq \delta$ 內；(ii) 在 $v \mapsto \ell + h - v$ 下對稱；(iii) 除尖端外處處切線連續，尖端處曲線折返（尖點）。*



圖 6
大括號外形

第 8 節　區域標籤與點標籤

標籤用來命名已繪製的填色區域或點。標籤圖層會在所有其他圖層之後，由延後處理程序繪製，因此能判斷必須避開哪些物件。點標籤先於區域標籤配置；每個已配置的標籤，都會成為後續標籤的障礙物。

8.1　區域標籤

RegionLabelLayer

```
RegionLabelLayer(region, text, short_text=None, placement="auto",
style=None,
                 stroke=Stroke(width=0.8), *, role="text", ...)
```

新增：*0.2.0*

為填色區域命名；區域可以是 `FillLayer` 的 id，也可以是至少由三點組成的多邊形。`stroke` 設定引線標註的引線樣式。

`placement="auto"`

若文字放得下，就以極點為中心置於區域內（參見定義 17）；若長文字放不下，再嘗試 `short_text`。兩者都放不下時，以 `text` 作引線標註。

`placement="inside"`
`placement="callout"`

與 `"auto"` 相同，但兩段文字都放不下時，仍會把 `short_text`（或 `text`）置中於極點。一律作引線標註：區域外不加框的文字，以及從極點拉到文字的細引線。

「文字放得下」是指：文字矩形置中於極點、每邊留白 2 pt 後，仍位於區域內（參見定理 14），且不碰到任何線條、標記、文字或其他區域。圖 7 同時呈現這兩種結果。

8.2　點標籤

PointLabelLayer

```
PointLabelLayer(point, text, style=None, *, role="text", ...)
```

新增：*0.3.0*

在點的旁邊加上文字為它命名，不使用引線。點的*足跡*是該處畫有標記時的標記，否則就是點本身。

標籤會在足跡周圍嘗試 16 個方向，依序與足跡邊緣間隔 4、7、11、16 pt，由近而遠。同一間隔內，依定義 21 的順序嘗試各方向（參見圖 8），採用第一個不遮蓋任何物件的位

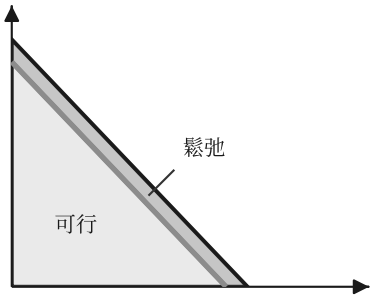


圖 7
區域內標籤與引線

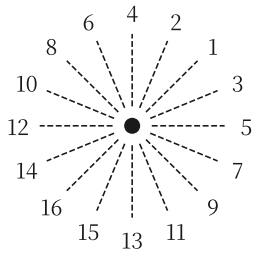


圖 8
方向的嘗試順序

置。若沒有合適的位置，就選擇違規數最少者；違規數相同時，採用較早嘗試的位置，並發出指名該圖層的 `LayoutWarning`。若點嚴格位於某區域內（距邊緣超過 1.5 px），標籤應位於該區域內，因此不會將它視為障礙物。

定義 21 (候選順序) 將方向自 $+x$ 起逆時針編號為 $k = 0, \dots, 15$，角度為 $2\pi k/16$。點標籤依鍵值 $(\min(s, 16 - s), [s > 8])$（$s = (k - 2) \bmod 16$）遞增的順序嘗試：依與右上方向 $k = 2$ 的角距離，角距離相同的兩個方向則逆時針者優先。

8.3　微調標籤

`Nudge`　`Nudge(dx=0.0, dy=0.0)`

`nudge_from`　`nudge_from(*, top=None, bottom=None, left=None, right=None, font_size=None) -> Nudge`

新增：*v1.0.0rc6*

在繪製器配置好標籤之後，再微調單一標籤的位置。`Nudge(dx, dy)` 使標籤向右移 `dx` 點、向上移 `dy` 點；`nudge_from` 以方向建立同樣的位移，其中 `dy = top - bottom`、`dx = right - left`。長度可以是以點為單位的數字，或帶單位的字串（`"10pt"`、`"0.5em"`、`"2mm"`、`"0.1in"`）；使用 `em` 時須給 `font_size`。`parse_length` 把單一長度轉成點。點標籤、區域標籤、大括號標籤與文字圖層都接受 `nudge=`；引線會跟著移動後的標籤，之後的標籤也會避開它。沒有微調時，輸出不變。

```
from mosaickit import Canvas, CanvasSpec, PointLabelLayer, nudge_from
canvas = Canvas(CanvasSpec(x_range=(0, 10), y_range=(0, 10)))
```

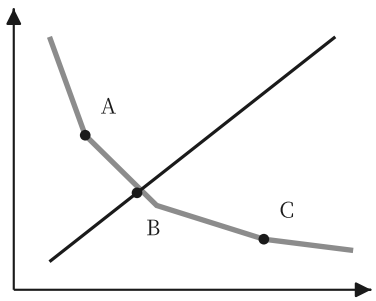


圖 9
點旁的點標籤

```
canvas.add(PointLabelLayer((5, 5), "$E$", nudge=nudge_from(top="3pt",
right="2pt")))
```

8.4　硬性限制

兩種標籤都遵守同一條規則：不遮蓋任何物件。障礙物包括座標軸上已繪製的內容：路徑的線段、標記與文字的矩形，以及填色區域的多邊形。以下搜尋函式位於 `mosaickit.layout.placement`；除 `place_beside` 外，也都由 `mosaickit.layout` 匯出。它們與幾何模組一樣，以顯示像素為單位運算；`scale` 則將以點為單位的間距換算為像素。

`Obstacles`　`Obstacles(segments=(), rects=(), polygons=())`

`Placement`　`Placement(rect, leader, violations)`

標籤必須避開的障礙物，以及標籤的配置結果：矩形、引線線段（不畫引線的標籤為 `None`），以及違反的限制數。`Obstacles.extended(segments=..., rects=...)` 會加入剛配置的標籤。

定義 22 (違規數) 對候選矩形 R、障礙物 O、繪圖區界線 B 與多邊形集合 $\mathcal{P}$，*違規數*為

$$V(R) = [R \not\subseteq B] + \#\{s \in O_{\text{seg}} : R \cap s \neq \emptyset\} + \#\{Q \in O_{\text{rect}} : R \cap Q \neq \emptyset\} + \#\{P \in \mathcal{P} : R \cap \overline{P} \neq \emptyset\},$$

其中 $[\cdot]$ 在條件成立時為 1，否則為 0。$V(R) = 0$ 時，稱候選位置*不遮蓋任何物件*。

各項分別由引理 10、`Rect.intersects` 與推論 15 精確計算。

8.5　引線標註

`place_callout`　`place_callout(polygon, size, obstacles, bounds, *, scale=1.0) -> Placement`

新增：*0.2.0*

在區域周圍搜尋能容納指定大小引線標註的位置。搜尋從極點 o 沿 16 個方向 r 進行：把候選矩形放在 `ray_exit(o, r, P)` 的距離之外，再間隔 12、24 或 40 pt（近環），並將面向極點的邊或角對齊，使矩形向遠離區域的方向延伸。引線從極點拉向矩形上的最近點（參見引理 6），並在距該點 2 pt 處停止。候選位置的代價以 $V(R)$ 為基礎（區域本身也列入多邊形）：若中心不在開闊空間中，加一；引線離開自身區域後，每穿過一條線、一段文字或一個其他區域，加一；每位於一個交叉點之外（參見定義 23），再加一。

定義 23 (開闊空間與交叉點) *開闊空間*是在繪圖區 4 pt 網格上、面積至少占十分之一的連通空白區域的聯集；較小的空白區域稱為*口袋*，例如兩個陰影區域間未填色的窄帶，文字放在那裡看起來像在為口袋命名。區域的*交叉點*是兩條障礙線段真正交叉（各自嚴格穿過對方）之處，且位於區域邊界上或距邊界 1.5 px 以內。從極點 o 看，若 $(p - x) \cdot (x - o) > 0$，則稱點 p 位於 x *之外*。

越過交叉點後，線條會向遠離區域的方向分開。若把標籤放在那裡，看起來會貼近線條的延伸方向，而非所命名的區域。交叉點本身由方向測試求得。

引理 24 (交叉點) 若 $[a,b]$ 與 $[c,d]$ 真正交叉，則它們恰交於一點 $a + t(b-a)$，其中

$$t = \frac{(c-a)\times(d-c)}{(b-a)\times(d-c)} \in (0,1),$$

而 $u \times v = u_x v_y - u_y v_x$。

命題 25 (引線標註的選擇) 以鍵值（代價, 引線長度）依字典序排列候選位置，並逐方向、由近到遠的距離列舉。若近環中有代價為 0 的候選，`place_callout` 回傳其中引線最短的第一個；否則再嘗試遠環（60 與 90 *pt*），回傳兩環中鍵值最小的第一個候選。結果只取決於引數。

回傳的代價不為零時，繪製器仍會在該處畫出引線標註，並發出指名該圖層的 `Layout Warning`；放大畫布或縮短文字，通常就能消除警告。

`fits_inside`	`fits_inside(polygon, size, obstacles, bounds) -> Rect | None`
`place_point_label`	`place_point_label(footprint, size, obstacles, bounds, *, scale=1.0) -> Placement`
`place_beside`	`place_beside(anchor, size, *, axis, direction, reach, obstacles, bounds, scale=1.0)`

其他搜尋包括：置中於極點的矩形，若放得下且除自身區域外不碰到任何物件，就回傳該矩形；前述的點標籤搜尋；以及大括號尖端旁的搜尋。最後一種搜尋會嘗試尖端外 3 到 45 pt 的間隔，並沿跨距以 3 pt 為步長滑動，最多滑動 `reach`，且位移較小者優先。`place_beside` 回傳矩形與其違規數。若該處沒有空間容納大括號標籤，繪製器會改從尖端嘗試引線標註，並保留違規較少的結果。

第 9 節　樣式與顏色

9.1　稀疏樣式

每個樣式都是凍結的 dataclass，且所有欄位都可以是 `None`。`None` 表示繼承：欄位值取自較低優先序的樣式（詳見第 10 節）。假值不等於 `None`，因此 `opacity=0`、`width=0` 與 `LegendStyle(visible=False)` 都是明確的覆寫。

定義 26 (稀疏合併) 設 a、b 為同型別、欄位集合為 F 的樣式。樣式 $a \triangleright b$（`a.merged_over(b)`）對每個 $\varphi \in F$ 滿足

$$(a \triangleright b)_\varphi = \begin{cases} a_\varphi & \text{if } a_\varphi \neq \text{None} \\ b_\varphi & \text{otherwise.} \end{cases}$$

空樣式 ε 的每個欄位都是 `None`。樣式組逐槽合併，`None` 槽視同 ε。

命題 27 (合併構成么半群) 對同型別的樣式 a, b, c，(*i*) 結合律為

$$(a \triangleright b) \triangleright c = a \triangleright (b \triangleright c);$$

(*ii*) 空樣式是單位元，

$$\varepsilon \triangleright a = a \triangleright \varepsilon = a;$$

(*iii*) 合併具冪等性，

$$a \triangleright a = a.$$

逐欄來看，$a_1 \triangleright \cdots \triangleright a_n$ 取第一個不是 *None* 的值。

因此，多個樣式無論如何分組合併，結果都一樣。可將它們視為一份優先順序清單：某欄位的值取自第一個設定該欄位的來源。

`SparseStyle` `style.merged_over(base)`
定義 26 的合併。不同型別的樣式引發 `TypeError`。

9.2　樣式型別

`Stroke` `Stroke(color=None, width=None, dash=None, arrow=None, opacity=None)`
線條樣式。`width` 以點為單位；`dash` 為 `DashStyle`（`SOLID`、`DASHED`、`DOTTED`、`DASHDOT`）；`arrow` 為 `ArrowStyle`（`OPEN`、`TRIANGLE`、`FANCY`、`WEDGE`），畫在圖層的 `ArrowPlacement`（`START`、`END`、`BOTH`）處。

`DashStyle` `DashStyle.SOLID, DASHED, DOTTED, DASHDOT`
`Stroke` 的線條型態；也接受簡寫 `-`、`--`、`:` 與 `-.`。

`SOLID` 實線。

DashStyle
`DASHED` 虛線。

DashStyle
`DOTTED` 點線。

DashStyle
`DASHDOT` 點劃線。

DashStyle

`ArrowStyle` `ArrowStyle.OPEN, TRIANGLE, FANCY, WEDGE`
`Stroke` 的箭頭樣式。`SIMPLE` 是 `OPEN` 的另一個名稱，也接受 Matplotlib 寫法 `->` 與 `-|>`。

`OPEN` 開口箭頭。

ArrowStyle
`TRIANGLE` 實心三角形。

ArrowStyle

FANCY　彎曲、漸細的箭頭。

ArrowStyle

WEDGE　細楔形箭頭。

ArrowStyle

ArrowPlacement　`ArrowPlacement.START, END, BOTH`
圖層的箭頭畫在哪裡。

START　在第一個點。

ArrowPlacement

END　在最後一個點。

ArrowPlacement

BOTH　兩端都畫。

ArrowPlacement

Fill　`Fill(color=None, opacity=None, hatch=None)`
區域內部的填色樣式；`hatch` 為 Matplotlib 的網紋樣式，例如 `"//"`，`""` 表示無網紋。

Marker　`Marker(color=None, size=None, shape=None, opacity=None, edge_color=None, edge_width=None)`
點標記樣式。`size` 是以平方點計的面積，與 Matplotlib 的 `scatter` 相同（36 對應 6 pt 圓點）；`shape` 為 Matplotlib 標記，例如 `"o"`、`"s"` 或 `"X"`。

TextStyle　`TextStyle(color=None, size=None, font=None, weight=None, opacity=None, rotation=None)`
文字樣式。`size` 以點為單位，`font` 是通用字型（`"sans"`、`"serif"` 或 `"mono"`，見第 9.3 節），`weight` 是 1 到 1000 的整數、`"normal"` 或 `"bold"`，`rotation` 為逆時針旋轉的角度（度）。欄位請以名稱傳入。

9.3　字型

文字由 glyphkit 依字型檔排版。三種通用字型指向字型檔：`"sans"`（預設）、`"serif"` 與 `"mono"`。MosaicKit 預設用 DejaVu Sans、DejaVu Serif 與 DejaVu Sans Mono，也就是 Matplotlib 附的檔案。它**不會搜尋系統字型**，其他字型一律要自己給檔案。

TextFonts　`TextFonts(sans=None, serif=None, mono=None)`
取代通用字型的字型檔。每個欄位可給一個檔案或一份清單；需要的字面都要列出，例如一般與粗體兩個檔案。樣式的 `weight` 會選最接近的字面，檔案順序則是第一個檔案缺字時依序嘗試的順序。沒給的欄位沿用 DejaVu 預設。

MathFont　`MathFont(path, italic=True)`
OpenType 數學字型，如 STIX Two Math、Libertinus Math 或 Fira Math，檔案要有 `MATH` 表。`italic` 讓單一拉丁字母與小寫希臘字母用斜體字母表，和 TeX 一樣。沒有數學字型時，`$...$` 內的數學式由 Matplotlib 排版。

```
from mosaickit import Canvas, CanvasSpec, Config, MathFont, TextFonts,
TextLayer, TextStyle, use_config
```

```
fonts = TextFonts(serif=["fonts/Pagella-Regular.otf", "fonts/Pagella-
Bold.otf"])
config = Config(fonts=fonts, math_font=MathFont("fonts/Pagella-
Math.otf"))

with use_config(config):
    canvas = Canvas(CanvasSpec(x_range=(0, 1), y_range=(0, 1)))
    canvas.add(TextLayer((0.5, 0.5), "$u = x^{0.5}$",
style=TextStyle(font="serif")))
```

TikZ 渲染器把 `sans`、`serif`、`mono` 對應到 `\sffamily`、`\rmfamily` 與 `\ttfamily`。

`LegendStyle` `LegendStyle(visible=None, location=None, frame=None, size=None)`
圖例樣式：包括 Matplotlib 的位置名稱（例如 `"best"` 或 `"upper right"`）、外框與字級。

建立樣式時會檢查大小、寬度與透明度：大小必須是有限的非負數，透明度必須在 $[0,1]$ 內，旋轉角度也必須是有限值。

表 4 基本預設值

樣式	值
`Stroke`	`grey-800`、寬 1.5、實線、透明度 1
`Fill`	`grey-200`、透明度 0.3、無網紋
`Marker`	`grey-800`、大小 36、`"o"`、透明度 1、邊框 `grey-800` 寬 0
`TextStyle`	`grey-900`、12 pt、DejaVu Sans、一般字重、透明度 1、不旋轉
`LegendStyle`	顯示、位置 `"best"`、無外框、10 pt

9.4　顏色

`Color` `Color(red, green, blue, alpha=1.0)`

`TRANSPARENT` `Color.from_hex("#RGB" | "#RRGGBB" | "#RRGGBBAA")`
不可變的 RGBA 顏色，各通道在 $[0,1]$ 內。`channels` 回傳四個值，`from_channels()` 建立顏色，`to_hex(include_alpha=None)` 寫出 `#RRGGBB`，當 `include_alpha` 為真、或預設情況下 alpha 不為 1 時加上 `AA`。`TRANSPARENT` 為 `Color(0, 0, 0, 0)`。

命題 28 (十六進位往返) *對每個由十六進位數字組成、形如 `#RRGGBB` 或 `#RRGGBBAA` 的字串 h，`Color.from_hex(h).to_hex(include_alpha=len(h) == 9)` 等於 h 的大寫形式。三位數的 `#RGB` 讀作 `#RRGGBB`。*

樣式的 `color` 或 `edge_color` 接受 `Color`、`"#hex"` 字串（立即解析），或其他任何字串，後者保留為色盤名稱。

9.5　色盤

`Palette` `Palette(name, colors)`

`DEFAULT_PALETTE`

具名的顏色表。值可以是 `Color` 或十六進位字串；`palette[name]` 用來查詢顏色，名稱不存在時會引發指名該色盤的 `ConfigurationError`；`name in palette` 用來檢查名稱是否存在。

新增：*0.2.0*

表 5 `DEFAULT_PALETTE`

名稱	值	預設主題中的用途
`grey-900`	`#222222`	文字
`grey-800`	`#333333`	線條、標記、座標軸
`grey-600`	`#666666`	座標軸註記
`grey-400`	`#999999`	輔助線
`grey-200`	`#CCCCCC`	填色
`grey-100`	`#E6E6E6`	
`white`	`#FFFFFF`	畫布背景
`blue`	`#01A2D9`	primary
`red`	`#E3120B`	secondary
`teal`	`#00887D`	accent

主題與樣式以名稱指定顏色。畫布建立繪製計畫時，才會從生效的色盤（`Config.palette`）中查詢這些名稱，因此繪製器只會收到具體顏色。只要在色盤中修改一次顏色，所有指定該顏色的角色都會隨之改變。色盤若缺少某個名稱，繪製時會引發 `ConfigurationError`，訊息會指明角色、樣式欄位與色盤。Python 的 `Palette` 會取代預設色盤，因此應以 `DEFAULT_PALETTE.colors` 為基礎建立，才能保留內建主題所用的名稱：

```
from mosaickit import DEFAULT_PALETTE, Canvas, Config, Palette,
use_config

brand = Palette("brand", {**DEFAULT_PALETTE.colors, "blue": "#0072B2"})
with use_config(Config(palette=brand)):
    canvas = Canvas()          # primary now draws in #0072B2
```

第 10 節　主題與設定

10.1　樣式組與主題

`StyleBundle`

`StyleBundle(stroke=None, fill=None, marker=None, text=None, legend=None)`

每個槽位包含一個稀疏樣式。槽位內的型別錯誤時，會引發 `ConfigurationError`。`merged_over(base)` 會逐槽位合併（參見定義 26）。

`Theme`

`Theme(name, roles)`

包含主題名稱，以及從角色名稱到樣式組的不可變對應。角色名稱必須是非空、以點分隔的字串（`"axes"`、`"axes.note"`、`"mypkg.boundary"`）。`with_roles(**patch)` 回傳新主題，並將每個修補角色合併在原有的樣式組之上。由於關鍵字名稱不能含點，帶點

的角色要寫成 `with_roles(**{"mypkg.line": bundle})`。`resolve(role, fallback_category=...)` 僅根據主題解析角色。

表 6　`default` 主題

角色	樣式組
primary	線條 blue
secondary	線條 red
accent	線條 teal
guide	線條 grey-400、虛線
axes	線條 grey-800、寬 1
axes.note	文字 9 pt、grey-600
canvas	填色 white、透明度 1（背景）

其餘的值都來自表 4 的基本預設值。此主題為 `mosaickit.themes.default`。

10.2　角色解析

解析圖層角色時，會依序查看該角色及其上層角色，最後才查看圖層型別的後備類別。這個過程逐欄進行；只有該角色與所有上層角色都未提供某欄的值時，才會使用後備類別的值。

定義 29 (角色鏈) 對後備類別為 φ 的角色 $\rho = \rho_1.\rho_2\cdots\rho_m$，其鏈為序列

$$\kappa(\rho) = (\rho_1\cdots\rho_m, \rho_1\cdots\rho_{m-1}, \ldots, \rho_1, \varphi),$$

由最具體到最一般，重複的鍵只保留第一次出現。

例如，文字圖層的 `"axes.note"` 角色鏈為（axes.note, axes, text）。後備類別包括 primary（路徑、群組）、region（填色）、point（標記）、text（文字、標籤、座標軸標記、註記、大括號）、annotation（箭頭）與 legend。

定理 30 (解析順序) 設某圖層的自帶樣式為 E、角色鏈為 $\kappa = (k_1, \ldots, k_n)$，畫在角色覆寫為 C 的畫布上。再設生效設定的角色覆寫為 G、主題角色為 T，並令 D 為基本預設值。則圖層解析後樣式的每個欄位，是序列

$$E,\quad C[k_1], \ldots, C[k_n],\quad G[k_1], \ldots, G[k_n],\quad T[k_1], \ldots, T[k_n],\quad D$$

中第一個不是 *None* 的值，其中不存在的角色視為空樣式組。

推論 31 (覆寫優先於具體程度) 畫布或設定對一般角色的覆寫，優先於主題對較具體角色的設定：若 G[text] 設定了文字大小，且定理 *30* 序列中在它之前沒有來源設定大小，則角色為 *axes.note* 的文字圖層採用該大小，而非主題的 *9 pt*。

本手冊的圖便運用了推論 31：圖中同時覆寫 `text`、`axes` 與 `axes.note` 的文字大小，因為只覆寫 `text` 也會放大註記。

10.3　註冊領域角色

`ThemeRegistry`　`ThemeRegistry()`

主題登錄表，建立時已含有 `default`。`register(theme)` 加入主題（名稱已被使用時引發 `ConfigurationError`），`get(name)` 查詢主題，`register_roles(name, roles)` 則把帶點的領域角色合併到已登錄的主題中，並回傳該主題。不帶點的角色會引發 `ConfigurationError`，因此領域套件無法遮蔽內建角色。

`RolePack`　`expand_roles(pack) -> dict[str, StyleBundle]`

`expand_roles`

以型別定義領域套件角色的方式。`RolePack` 是帶有類別變數 `_namespace` 的 dataclass；每個不為 `None` 的欄位都會轉成 `namespace.field` 角色，欄位名稱中的底線則轉為點。

```
from dataclasses import dataclass
from typing import ClassVar
from mosaickit import Stroke, StyleBundle, Theme, expand_roles
from mosaickit.themes import default

@dataclass(frozen=True)
class MyRoles:
    _namespace: ClassVar[str] = "mypkg"
    line: StyleBundle | None = StyleBundle(stroke=Stroke(color="blue",
width=2))
    line_dashed: StyleBundle | None = None

roles = expand_roles(MyRoles())        # {"mypkg.line": ...}
mytheme = Theme("mypkg", {**default.roles, **roles})
```

10.4　設定

`Config`　`Config(theme=default, canvas_spec=CanvasSpec(), renderer="matplotlib", role_overrides={}, palette=DEFAULT_PALETTE, math_font=None, fonts=None)`

新畫布的執行期預設值：主題、規格、繪製器（名稱或 `Renderer`）、在主題之後套用的角色覆寫（定理 30 中的 G），以及色盤。`math_font` 是 `MathFont`，`fonts` 是 `TextFonts`（第 9.3 節）。

`use_config`　`with use_config(config): ...`

讓 `config` 在區塊內成為生效的設定。此值存於 context variable，因此每個執行緒與非同步工作都有各自的設定；傳給 `Canvas` 的引數一律優先。

`Config.load`　`Config.load(path)`

`Config.from_dict`　`Config.from_dict(data, *, source="<dict>")`

讀取嚴格的 TOML 設定。最上層的鍵為 `theme`（只能是 `"default"`；自訂主題須在 Python 中傳入）、`canvas`（`CanvasSpec` 的欄位）、`renderer`（只能是 `"matplotlib"`）、`palette`、`styles`、`math`（`font` 為數學字型的路徑，另有 `italic`）與 `fonts`（`sans`、`serif`、

mono，各是一份字型檔清單）。相對路徑以設定檔所在資料夾為準。其他鍵、未知的樣式、錯誤的值或無法讀取的檔案，都引發指明檔案與鍵的 ConfigurationError。

```toml
[canvas]
x_range = [0, 20]
dpi = 150

[palette]
blue = "#0072B2"      # override a default color
accent = "#984EA3"    # add a new name

[styles."mypkg.boundary".stroke]
color = "accent"      # a palette name or a "#hex" value
width = 2
dash = "dashed"
```

[palette] 表疊加在 DEFAULT_PALETTE 之上，因此在此覆寫 blue 會改變 primary 以及所有指名 blue 的地方。樣式的 color 與 edge_color 可以是色盤名稱；未知名稱在載入檔案時（任何畫布繪製之前）就引發指明檔案與鍵的 ConfigurationError。

10.5　EconViz 套件共用的設定檔

每個 EconViz 圖形套件都讀取同一個檔案 econ-viz.toml。最上層放所有圖形共用的設定：base 是起始的內建主題；[font] 有 text（通用字型）與 math（數學字型檔）；[fonts] 是取代通用字型的字型檔；[color] 有 background、axis 與 label。每個套件把自己的設定放在以套件命名的一個表格裡，例如 [principle] 或 [utility]，並略過其他套件的表格。相對路徑從檔案所在的資料夾讀取。

```toml
base = "manual"

[font]
text = "serif"
math = "fonts/Pagella-Math.otf"

[fonts]
serif = ["fonts/Pagella-Regular.otf", "fonts/Pagella-Bold.otf"]

[color]
background = "#ffffff"

[principle.color]
demand = "#8c8c8c"
```

Schema　`econ_toml.Schema(package, base, fields={}, patterns={}, shared={}, ...)`

新增：*v1.0.0rc6*

套件不自己解析這個檔案，而是在 mosaickit.econ_toml 中宣告一個 Schema：內建主題（base）、自己表格的各個鍵與每個鍵設定的屬性、名稱自由的表格（patterns），以及使用的共用設定。mosaickit 負責讀檔、檢查每個值、以檔案所在資料夾解析相對路徑，並指出錯誤所在的檔案與表格；鍵名拼錯時還會建議最接近的鍵。未知的最上層鍵會引發錯誤。Schema.config_class() 回傳該套件的 Config 類別，Schema.template() 寫出其表格附註解的區段。

econ_toml.load　`econ_toml.load(path=None) -> Mapping`

econ_toml.use　`econ_toml.use(path=None) -> None`

新增：*v1.0.0rc6* `load` 由同一個檔案建立所有已註冊套件的設定。`use` 做同樣的事，並為每個宣告了 `econviz.config` 進入點的已安裝套件啟用設定。

`get_tokens` `mosaickit.themes.get_tokens(name) -> ThemeTokens`

`list_shared_themes` `mosaickit.themes.list_shared_themes() -> tuple[str, ...]`

新增：*v1.0.0rc6* 每個 EconViz 套件都提供的主題有 `manual`、`default`、`colorblind`、`monochrome`、`nord`、`paper` 與 `presentation`。`ThemeTokens` 為 `ROLES` 中每個具名角色（`ink`、`primary`、`secondary`、`constraint`、`accent`、各區域填色等）指定顏色，另有三個尺寸倍率：`line_scale`、`marker_scale` 與 `label_scale`。套件只需把自己的元素對應到這些角色一次，同一個主題名稱在每個套件中就呈現相同的外觀。

第 11 節　參數與網格

11.1　參數與運算式

圖層座標不一定是數字，也可以是由具名參數組成的運算式。此時，場景描述一族圖形，每組參數值對應一張圖；綁定參數就是從中選出一張。

`Parameter` `Parameter(name, value_type=None)`

具名的佔位符。設定 `value_type` 後，綁定值必須是該型別的實例；所有綁定值都必須可雜湊。`values(seq)` 回傳 `ParameterValues`，並立即檢查每個值。

`Expression` `Constant(value)`

`Constant`

不可變的運算式樹。參數與常數可以透過 `+`、`-`、`*`、`/`、`**`、一元 `-`，以及比較運算 `<`、`<=`、`>`、`>=` 與 `equals()` 組合；一般數字會包裝成常數。運算式採結構相等：`==` 比較兩棵運算式樹，要建立比較運算式時應改用 `equals()`。把運算式當成真假值使用會引發 `BindingError`；使用前須先求值。`evaluate(bindings)` 根據參數到值的對應計算結果；`free_parameters()` 回傳樹中所有參數的集合。

定義 32 (運算式與綁定) *運算式*是常數、參數，或對運算式 e_1, e_2 與運算 $\circ$ 而言的 $e_1 \circ e_2$。其自由參數滿足

$$\operatorname{free}(c) = \emptyset, \quad \operatorname{free}(p) = \{p\},$$

以及

$$\operatorname{free}(e_1 \circ e_2) = \operatorname{free}(e_1) \cup \operatorname{free}(e_2).$$

綁定 β 是從參數到值的有限對應。若 $\operatorname{free}(e) \subseteq \operatorname{dom}\beta$，則 $\operatorname{bind}(e, \beta)$ 為值 $e(\beta)$；若 e 是未綁定參數，則為 e 本身；其餘情況為

$$\operatorname{bind}(e_1, \beta) \circ \operatorname{bind}(e_2, \beta).$$

一般值 v 沒有自由參數，求值結果為其本身。

定理 33 (部分綁定) *對每個運算式 e 與綁定 β，* (i)

$$\operatorname{free}(\operatorname{bind}(e, \beta)) = \operatorname{free}(e) \setminus \operatorname{dom}\beta;$$

(ii) *對每個滿足*

$$\operatorname{dom}\gamma \cap \operatorname{dom}\beta = \emptyset$$

以及

$$\operatorname{dom}\gamma \supseteq \operatorname{free}(e) \setminus \operatorname{dom}\beta$$

的綁定 γ，都有

$$\operatorname{bind}(e, \beta)(\gamma) = e(\beta \cup \gamma).$$

推論 34 (分階段綁定) *對定義域不相交的綁定 β_1, β_2，* $\operatorname{bind}(\operatorname{bind}(e, \beta_1), \beta_2)$ *與* $\operatorname{bind}(e, \beta_1 \cup \beta_2)$ *有相同的自由參數，且在這些參數的每個綁定下有相同的值。特別地，* `canvas.bind(p, 1).bind(q, 2)` *與* `canvas.bind({p: 1, q: 2})` *畫出相同的圖。*

`bind` 會走訪 tuple、mapping 與所有 `mosaickit` dataclass，因此能一次綁定整個場景；圖層的 `model` 則保持不變。若繪製的場景仍含自由參數，會引發指名這些參數的 `BindingError`。

```
from mosaickit import Canvas, Parameter, TextLayer

x = Parameter("x", value_type=float)
template = Canvas().add(TextLayer((x, 2 * x + 1), "moving"))
```

```
frame = template.bind(x, 3.0)
print(frame.snapshot().layers[0].position)    # (3.0, 7.0)
```

11.2　網格

CanvasGrid　`CanvasGrid(cells, rows=None, cols=None, shape=None, links=())`

Span　`Span(canvas, rows=1, cols=1)`

在同一張圖中放入多個畫布。`cells` 可以是由畫布、`Span` 與 `None`（空格）組成的平面清單，並逐列填入；也可以是列的清單，此時允許跨列，且每列必須涵蓋所有欄。`shape=(rows, cols)` 等同於同時傳入兩者。每個畫布都會複製，因此之後修改原畫布不會影響網格。跨格重疊、跨格超出邊界、格子過多，或混用平面與巢狀格子，都會引發 `ConfigurationError`。`render()` 與 `save()` 的用法與畫布相同。

命題 35 (推斷的網格形狀) *對 $n \geq 1$ 個一般格子組成的平面清單，若未給 `rows` 與 `cols`，網格有 $c = \lceil \sqrt{n} \rceil$ 欄、$r = \lceil n/c \rceil$ 列。此時 $rc \geq n$、$r \leq c$，且空格少於 c 個，全部位於最後一列。*

只給 `cols` 時 $r = \lceil n/c \rceil$；只給 `rows` 時 $c = \lceil n/r \rceil$。

Layout　`CanvasGrid.from_layout(canvases, layout)`

CanvasGrid.from_layout

常見的排列：`SINGLE`、`STACKED`（兩列）、`SIDE_BY_SIDE`、`GRID_2X2`、`GRID_3X3`，以及三個畫布的 `TOP_TWO_BOTTOM_ONE` 與 `TOP_ONE_BOTTOM_TWO`，其中單獨的畫布橫跨兩欄。

CanvasGrid.sweep　`CanvasGrid.sweep(template, values, *, cols=None)`

`ParameterValues` 的每個值一格，每格是綁定該值的範本。

GridLink　`GridLink(start_cell, start, end_cell, end, role="link", stroke=None)`

新增：*0.5.0*

從某格的 `start` 畫到另一格 `end` 的直線；兩點分別以所在格子的資料座標表示。直線畫在整張圖上，會橫越格子間的空隙。格子依配置順序編號。樣式先在起點格子的主題中解析 `role`，再疊上 `stroke`。格子編號超出範圍時，建立網格就會引發 `ConfigurationError`。

```
from mosaickit import Canvas, CanvasGrid, GridLink, MarkerLayer,
Parameter, Stroke

shift = Parameter("shift", value_type=float)
template = Canvas().add(MarkerLayer([(5, 2.5 + shift)]))
cells = [template.bind(shift, v) for v in (0.0, 2.0, 4.0)]
link = GridLink(0, (5, 2.5), 2, (5, 6.5), stroke=Stroke(dash="dashed"))
CanvasGrid(cells, cols=3, links=[link]).save("sweep.pdf")
```

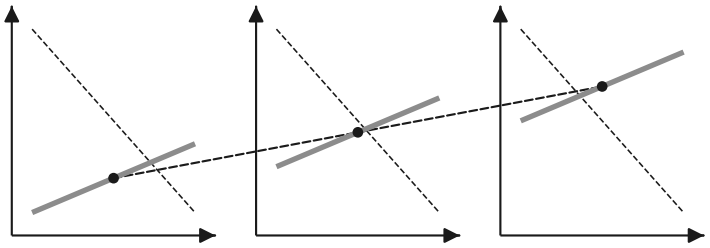


圖 10
三個值的掃描

第 12 節　繪製

12.1　繪製器

Renderer　`name: str`

`render(scene, context) -> Any`

`save(result, target, options) -> list[Path]`

所有後端都必須實作的協定。繪製器接收不可變的場景與私有的繪製情境（規格、主題、覆寫、色盤、快取與綁定），並回傳由 `save` 寫出的結果。網格另需 `render_grid`；繪製器若沒有實作它，會引發 `RenderError`。內建後端仍在演進，因此繪製計畫與情境維持私有。

RendererRegistry　`RendererRegistry()`

`register(renderer)` 以 `name` 登錄後端；若物件缺少 `name`、`render` 或 `save`，或名稱已被使用，會引發 `RenderError`。`get(name)` 回傳已登錄的繪製器，或載入內建繪製器。唯一的內建繪製器是 `"matplotlib"`，在第一次使用時才匯入 (Hunter, 2007)。

12.2　Matplotlib 繪製器

繪製畫布時，會先建立繪製計畫：綁定場景（有自由參數時引發 `BindingError`）、展開群組、依 `z_index` 排序圖層、解析每個圖層的樣式（參見定理 30），並把色盤名稱換成具體顏色。圖形採用畫布的尺寸與 DPI，背景採用 `canvas` 角色的填色；座標軸範圍來自規格，並會關閉 Matplotlib 本身的軸線。mosaickit 中的座標軸是圖層。

接著，各圖層型別的建構函式會依序繪製圖層。工作需參照其他所有內容的圖層型別（圖例、點標籤、區域標籤、邊欄文字、跨距大括號），會在後續的延後處理程序中繪製。各程序依登錄順序執行，一次接收該型別的所有圖層。

register_builder　`register_builder(layer_type, builder)`

register_pass　`register_pass(layer_type, run)`

新增：*0.2.0*

這兩個函式位於 `mosaickit.rendering.matplotlib`，可在不修改 mosaickit 的情況下，讓繪製器認得新的圖層型別。建構函式以 `builder(ax, resolved)` 呼叫，並回傳 Matplotlib artist；處理程序以 `run(ax, layers, context)` 呼叫，`PassContext` 的 `handles` 會將圖層 id 對應到圖例用的 artist。`resolved.layer` 是圖層，`resolved.style` 是解析後的 `StyleBundle`。查詢依方法解析順序進行，因此子類別會繼承父類別的登錄。

圖層透過類別變數 `style_slots` 宣告各槽位的自帶樣式存於哪個欄位，再以 `fallback_category` 宣告作為最後依據的角色。

區域標籤的引線標註會避開座標軸上的所有內容，包括第三方登錄的建構函式所畫的物件。

12.3　結果與儲存

`SaveOptions`　`SaveOptions(transparent=False, expand=True)`

設定結果的寫出方式；`canvas.save(path, **options)` 會把關鍵字引數傳到這裡。`transparent` 去除背景。`expand=True` 時，儲存會把畫布擴大到恰好容納畫到邊緣外的內容（例如邊欄文字），另加 4 pt 留白。儲存時絕不會裁切畫面，因此原本就放得下的圖，會維持 `CanvasSpec` 指定的尺寸。`expand=False` 則保持指定尺寸。

```
result = canvas.render()
try:
    result.show()          # an interactive window
finally:
    result.close()
```

`render()` 回傳 `MatplotlibResult`，提供 `figure`、`axes`、`show()`、`save(target, **options)` 與 `close()`。`canvas.save()` 會自行關閉暫時結果。檔案格式由副檔名決定：`.png`、`.pdf` 或 `.svg`；其他副檔名會引發 `RenderError`。網格中每格大小相同：寬度取**各畫布寬度除以所跨欄數**的最大值，高度取**各畫布高度除以所跨列數**的最大值；DPI 取各畫布中的最高值。

12.4　快取

`RenderCache`　`RenderCache(max_entries=256)`

`CacheKey`

有容量上限且執行緒安全的最近最少使用快取，用來存放解析後的圖層，並以圖層 id、綁定、模型、規格與樣式為鍵。除非以 `cache=` 傳入快取，否則每次繪製或網格工作都會建立自己的快取，因此工作之間不會意外共用狀態。要在多個工作間重用結果，請傳入同一個快取。無法雜湊的模型會繞過快取直接繪製；每種模型型別在每個快取中只會發出一次 `CacheBypassWarning`。模型請使用凍結的 dataclass。`clear()` 清空快取。

附錄 A　證明

本附錄依各章出現的順序，證明其中的引理、命題、定理與推論。以下假設精確算術。程式以浮點數計算相同的量，因此若某配置與退化情形的差異僅在捨入誤差範圍內（例如點落在邊上，或三點幾乎共線），程式可能將它判定為任一種結果。平面向量 u, v 的外積記為 $u \times v = u_x v_y - u_y v_x$。

A.1　排列與軌道

定理 2 的證明： 變數代換。令 $o_k = \sum_{j<k}\big(s_j + g\big) + s_k/2$、$y_k = x_k - o_k$、$z_k = c_k - o_k$。由於 $o_{k+1} - o_k = (s_k + s_{k+1})/2 + g$，定義 1 的限制條件即 $y_1 \le \cdots \le y_n$，目標函數為 $\sum_k (y_k - z_k)^2$。因此問題是在最小平方意義下，找出最接近 z 的非遞減向量。

程式的計算。考慮從第 i 項開始、起點為 S_B 的連續項目群 B。對於 $k \in B$，程式將它放在

$$x_k = S_B + \sum_{i \le l < k} (s_l + g) + s_k/2 = S_B + o_k - O_i$$

其中 $O_i = \sum_{l<i}(s_l + g)$。因此 $y_k = S_B - O_i =: Y_B$ 在 B 上為常數。程式取 S_B 為 $c_k - (o_k - O_i)$ 的平均，使 Y_B 成為 z_k 在 B 上的平均；新加入的單一項目則有 $Y = z_k$。群 A 與後續群 B 在 $S_A + \text{length}(A) + g \le S_B$ 時保持分開。由於 $\text{length}(A) + g = O_{i_B} - O_{i_A}$，此條件等價於 $Y_A \le Y_B$。因此，程式實作的就是 PAVA：把 z_k 加入為一個區塊；只要最後兩個區塊的平均遞減，就以它們的聯集（取其平均）取代。

不變式。(a) 相鄰區塊的平均非遞減：最後一對順序正確時合併即停止，較早的區塊對不受影響。(b) 在平均為 μ_B 的每個區塊 B 中，每個起始片段的平均至少為 μ_B。單一項目滿足 (b)。A 與 B 合併時 $\mu_A > \mu_B$，合併後的平均 μ 嚴格介於兩者之間。A 內的起始片段平均至少為 $\mu_A > \mu$。其他起始片段是 A 接上 B 的起始片段 B'；若 B' 為整個 B，其平均為 μ；否則 B 的其餘部分 B'' 由 B 的 (b) 知平均至多 $\mu_B < \mu$，而該片段是平均為 μ 的集合中 B'' 的補集，平均至少為 μ。

最佳性。目標函數嚴格凸、限制條件 $y_k - y_{k+1} \le 0$ 為線性，因此滿足 Karush–Kuhn–Tucker 條件的點即唯一的最小解 (Boyd & Vandenberghe, 2004)。對 $k = 0, \ldots, n$ 令 $\lambda_k = 2\sum_{l \le k}(z_l - y_l)$。平穩條件 $2(y_k - z_k) + \lambda_k - \lambda_{k-1} = 0$ 依構造成立。區塊內的偏差 $z_l - Y_B$ 總和為零，因此 λ 在每個區塊末端為零，包括 $\lambda_n = 0$。在由第 i 項起的區塊內，由 (b) 得 $\lambda_k = 2\sum_{l=i}^{k}(z_l - Y_B) \ge 0$。最後，$\lambda_k > 0$ 只發生在 k 與 $k+1$ 屬於同一區塊時，此時 $y_k = y_{k+1}$，限制條件取等號。加上 (a) 給出的可行性，所有條件都成立。 □

推論 3 的證明： (i) 依排序後的順序編號，並設 $i < j$。把第 i 到第 $j-1$ 條限制相加，得

$$x_j - x_i \ge \sum_{k=i}^{j-1}\big((s_k + s_{k+1})/2 + g\big) \ge \big(s_i + s_j\big)/2 + g$$

因為每項都非負，且首末兩項分別貢獻 $s_i/2$ 與 $s_j/2$。(ii) 若 c 是排列，它使目標函數為 0，因此是定理 2 的唯一最小解。(iii) 沿用該證明的記號，

$$\sum_{k \in B}(x_k - c_k) = \sum_{k \in B}(y_k - z_k) = |B|\, Y_B - \sum_{k \in B} z_k = 0$$

□

定理 5 的證明： 區間只在與某軌道上所有區間都不衝突時才加入該軌道，之後不再移動，因此結果是軌道指派。任何軌道指派都必須把 ω 個兩兩衝突的區間放在 ω 條不同軌道，所以至少需要 ω 條軌道。

反過來，記 $J = [a_J, b_J]$，並設區間依 a 遞增的順序給出。由定義 4，I 與 J 衝突恰好等價於半開區間 $[a_I, b_I + g)$ 與 $[a_J, b_J + g)$ 相交。設 I 被放在所用的最高軌道 k。I 到來時，每條軌道 $j < k$ 上都有與 I 衝突的區間 J_j。每個 J_j 較早到來，故 $a_{J_j} \le a_I$；又與 I 衝突，故 $b_{J_j} + g > a_I$。因此每個 $\left[a_{J_j}, b_{J_j} + g\right)$ 都含 a_I，而 $b_I - a_I + g > 0$ 使 $[a_I, b_I + g)$ 也含 a_I。有共同點的半開區間兩兩相交，所以 $J_0, \ldots, J_{k-1}, I$ 是 $k+1$ 個兩兩衝突的區間，$k + 1 \le \omega$。 □

A.2　矩形、線段與多邊形

引理 6 的證明：$|q - p|^2 = (q_x - p_x)^2 + \left(q_y - p_y\right)^2$，而 R 是 $[x_0, x_1]$ 與 $[y_0, y_1]$ 的乘積，因此兩個座標可分別最小化。在區間上，$t \mapsto (t - p_x)^2$ 嚴格凸，無限制的最小值在 p_x，所以它在 $[x_0, x_1]$ 上的最小值只在 $\min(\max(p_x, x_0), x_1)$ 取得；y 亦同。 □

引理 8 的證明： 將其他列減去第一列，得

$$\operatorname{orient}(a, b, c) = \det \begin{pmatrix} 1 & a_x & a_y \\ 1 & b_x & b_y \\ 1 & c_x & c_y \end{pmatrix}.$$

列的循環置換為偶置換，交換兩列為奇置換，由此得到對稱性。又

$$\operatorname{orient}(a, b, c) = (b - a) \times (c - a) = |b - a|\,|c - a| \sin\theta$$

其中 θ 為從 $b - a$ 到 $c - a$ 的有號角；它為正恰好在 $\theta \in (0, \pi)$，即 c 位於有向直線左側時，位於右側時為負，而在兩向量平行或其一為零，即三點共線時為零。 □

定理 9 的證明： 先說明：若 $\operatorname{orient}(c, d, p) = 0$ 且 p 位於 $[c, d]$ 的外接矩形內，則 $p \in [c, d]$。事實上，若 $c = d$，外接矩形就是點 c；否則由共線性，對某實數 t 有 $p = c + t(d - c)$，而在 $d - c$ 不為零的座標上，外接矩形條件迫使 $t \in [0, 1]$。

充分性。設 $d_1 d_2 < 0$ 且 $d_3 d_4 < 0$。$\operatorname{orient}(c, d, \cdot)$ 沿 $[a, b]$ 為仿射函數且變號，因此 $[a, b]$ 與直線 L_{cd} 交於一點；同理 $[c, d]$ 與 L_{ab} 交於一點。由於 a、b 位於 L_{cd} 兩側，兩直線不平行，恰有一個共同點 z；兩個交點都等於 z，因此 z 同時位於兩線段上。另一種情形是某個 $d_i = 0$ 且其端點位於另一線段的外接矩形內，由上述說明，該端點位於另一線段上。

必要性。設 z 為共同點。若 $d_1 d_2 > 0$，則 a、b 嚴格位於 L_{cd} 的同一側，整個 $[a, b]$ 亦然，與 $z \in L_{cd}$ 矛盾；所以 $d_1 d_2 \le 0$，同理 $d_3 d_4 \le 0$。若兩者皆為負則得證。否則某個 d_i 為零；不妨設 $d_1 = 0$，其餘情形對稱。若 $d_2 \ne 0$，$[a, b]$ 只在 a 與 L_{cd} 相交，故 $z = a$ 且 $a \in [c, d]$：d_1 的測試成立。若 $d_2 = 0$，四點共線，兩線段是同一直線上重疊的區間，其中一條的端點位於另一條內，因此四個測試之一成立。 □

引理 10 的證明： R 的邊都在 R 內，因此任一測試成立都給出共同點（參見定理 9）。反之，設 $[a, b]$ 與 R 相交但兩端點都不在 R 內。線段是連通的，同時含有 R 內與 R 外的點，因此與 R 的邊界相交，而邊界是四條邊的聯集；定理 9 會偵測到該邊。 □

引理 12 的證明：$t \mapsto |a + t(b-a) - p|^2$ 是凸二次函數，無限制的最小值在 $(p-a)\cdot(b-a)/\,|b-a|^2$；如引理 6 的證明，它在 $[0,1]$ 上的最小值位於該值的截斷處。 □

命題 13 的證明：記 $p = (x, y)$，取 $\varepsilon > 0$ 小於所有頂點高度與 y 的正差值 $|y_i - y|$，且小到使 $p_\varepsilon = (x, y+\varepsilon)$ 與 p 位於 $\mathbb{R}^2 \setminus \partial P$ 的同一連通分量（因 $p \notin \partial P$ 且各分量為開集，此為可行）。直線 $Y = y + \varepsilon$ 上沒有頂點，且對每個頂點，$y_i > y$ 若且唯若 $y_i > y + \varepsilon$。因此一條邊通過程式的高度測試，恰好等價於它穿過直線 $Y = y + \varepsilon$，交點橫座標為

$$X_\varepsilon = x_0 + (y + \varepsilon - y_0)(x_1 - x_0)/(y_1 - y_0)$$

程式比較的是 x 與 X_0。若 $x = X_0$，由於高度測試使 y 介於兩端點高度之間，點 (X_0, y) 位於該邊上，即 $p \in \partial P$，與假設矛盾；所以 $x \neq X_0$，由連續性，對夠小的 ε，$x < X_0$ 若且唯若 $x < X_\varepsilon$。因此程式計算的正是從 p_ε 向右、不經過任何頂點的射線所穿過的邊。每次穿越都使射線在內部與外部之間切換，而射線在最右方位於外部，所以計數為奇數恰好在 $p_\varepsilon \in \operatorname{int} P$，即 $p \in \operatorname{int} P$ 時 (Haines, 1994)。 □

定理 14 的證明：若 $R \subset \operatorname{int} P$，則 R 與 ∂P 不相交，因此沒有邊碰到它（參見引理 10），而它的角位於 $\operatorname{int} P$ 且不在邊界上，因此通過奇偶測試（參見命題 13）。反之，若沒有邊碰到 R，則 $R \cap \partial P = \emptyset$（參見引理 10）；特別是各角不在邊界上，通過測試即表示它們位於 $\operatorname{int} P$。R 是連通的且與 ∂P 不相交，因此位於 $\mathbb{R}^2 \setminus \partial P$ 的一個分量內；它含有一個位於 $\operatorname{int} P$ 的角，所以 $R \subset \operatorname{int} P$。 □

推論 15 的證明：若有一個角通過測試，它不在邊界上時位於 $\operatorname{int} P$（參見命題 13），否則位於 ∂P；無論哪種情形都有 $R \cap \overline{P} \neq \emptyset$。若有一條邊碰到 R，則 R 與 ∂P 相交。反之，設 R 與 $\overline{P}$ 相交。若它與 ∂P 相交，就有某條邊碰到它。否則 R 與 ∂P 不相交而與 $\operatorname{int} P$ 相交；由連通性得 $R \subset \operatorname{int} P$，其各角通過測試。 □

命題 16 的證明：設對某個 $t > T$，$p = o + tr \in \partial P$，位於邊 e 上。若 e 不平行於 r，程式對 e 解 $o + tr = a + u(b-a)$，得到這個 $t \geq 0$ 與 $u \in [0,1]$，因此 $T \geq t$，矛盾。若 e 平行於 r，它位於射線所在直線 ℓ 上。取包含 e、位於 ℓ 上的極大連續邊段。它不是整個 P（P 的頂點不全共線），所以邊段兩端都是與某條不在 ℓ 上、因而不平行於 r 的邊共用的頂點。邊段中相鄰的邊不會折返，因為從同一頂點沿 ℓ 同向出發的兩條邊會重疊，與 P 為簡單多邊形矛盾；因此邊段是 ℓ 上的一段，其在 r 方向的遠端是端點頂點 $v = o + t_v r$，且 $t_v \geq t > T$。相鄰的不平行邊含有 v（$u \in \{0,1\}$），因此 $T \geq t_v$，同樣矛盾。所以 $\{o + tr : t > T\}$ 與 ∂P 不相交；它連通且無界，因此位於外部。 □

A.3　不可及極點

引理 18 的證明：對任意集合 S，由三角不等式得 $|d(p,S) - d(q,S)| \leq |p - q|$，這就處理了 p、q 在同一側的情形。設 $p \in \overline{P}$、$q \notin \overline{P}$，並令 z 為從 p 到 q 的線段上，屬於 $\overline{P}$ 的最後

一點（$\overline{P}$ 為閉集，故存在）。緊接在 z 之後的點都在外面，因此 $z \notin \operatorname{int} P$，即 $z \in \partial P$。於是

$$|f(p) - f(q)| = d(p, \partial P) + d(q, \partial P) \le |p - z| + |z - q| = |p - q|$$

□

定理 19 的證明：程式精確計算 f：不在邊界上時，由命題 13 符號正確；在邊界上時距離為 0。記 b 為目前的最佳值；它只會增加，且始終是某個已求值中心的 f 值。

結束性。每次分割使半邊長 h 減半。取出 $h\sqrt{2} \le \varepsilon$ 的格子時，更新後 $b \ge f(c)$，因此其上界至多比 b 多 $h\sqrt{2} \le \varepsilon$，格子被捨棄而不分割。所以格子構成有限樹，每格各推入與取出一次。

不變式。外接矩形的每一點都位於某格的閉正方形內，該格要不在佇列中，要不在被捨棄當時滿足 $f(c) + h\sqrt{2} \le b + \varepsilon$。初始正方形從左下角起以短邊為步長鋪設，直到越過遠端邊緣，覆蓋了外接矩形；捨棄依捨棄規則保持不變式；分割把一個正方形換成覆蓋它的四個正方形。

結論。結束時佇列為空。設 p^* 為極點；它位於 $\overline{P}$ 中、外接矩形內，落在某個被捨棄、中心為 c、半邊長為 h 的格子內。由引理 18 與 $|p^* - c| \le h\sqrt{2}$，

$$f^* = f(p^*) \le f(c) + h\sqrt{2} \le b + \varepsilon \le f(q) + \varepsilon,$$

其中 q 為回傳的點，$f(q)$ 為最終的 b。若 $f^* > \varepsilon$，則 $f(q) > 0$，故 q 位於 $\overline{P}$ 內且與 ∂P 距離為正，即 $q \in \operatorname{int} P$。 □

命題 20 的證明：記半徑為 r，拉伸倍率為 $\sigma = \delta/2r \ge 1$。拉伸之前，曲線由以下部分組成：圓心 $(0, \ell + r)$、從 $-90°$ 到 $0°$ 的圓弧 A_1；從 $v = \ell + r$ 到 $v = m - r$ 的線段 $u = r$；圓心 $(2r, m - r)$、從 $180°$ 到 $90°$ 的圓弧 A_2；圓心 $(2r, m + r)$、從 $270°$ 到 $180°$ 的圓弧 A_3；從 $m + r$ 到 $h - r$ 的線段 $u = r$；以及圓心 $(0, h - r)$、從 $0°$ 到 $90°$ 的圓弧 A_4。（當 $r = (h - \ell)/4$ 時兩線段長度為零。）曲線起於 $(0, \ell)$、終於 $(0, h)$，A_2 與 A_3 交於 $(2r, m)$，拉伸 $(u, v) \mapsto (\sigma u, v)$ 把它送到 (δ, m)。

(i) 在 A_1 與 A_4 上 $u \in [0, r]$；在兩線段上 $u = r$；在 A_2 與 A_3 上 $u \in [r, 2r]$。拉伸後 $u \in [0, \delta]$。

(ii) 反射 $v \mapsto \ell + h - v$ 交換 A_1 與 A_4、A_2 與 A_3 的圓心，並把角度 θ 變為 $-\theta$，因此把 A_1 映到 A_4、A_2 映到 A_3，兩線段互換。

(iii) 沿角度遞增方向走圓弧時，運動方向為 $(-\sin\theta, \cos\theta)$；沿遞減方向時為 $(\sin\theta, -\cos\theta)$。在 A_1 終點（$\theta = 0$）與 A_2 起點（$\theta = 180°$）兩者都是 $(0, 1)$，即兩者間線段的方向；A_3 終點與 A_4 起點亦同。在尖端，A_2 以方向 $(1, 0)$ 抵達（$\theta = 90°$），A_3 以 $(-1, 0)$ 離開（$\theta = 270°$）：形成尖點。拉伸與最後的擺放都是可逆仿射映射，會帶著切線方向且保持其非零，因此連續性與尖點都得以保留。 □

A.4　標籤

引理 24 的證明： 共同點滿足 $a + tr = c + us$，其中 $r = b - a$、$s = d - c$。兩邊與 s 作外積可消去 u：$t(r \times s) = (c - a) \times s$。真正交叉使 a、b 嚴格位於 L_{cd} 兩側，因此兩直線不平行，$r \times s \neq 0$，交點唯一。由於 $\mathrm{orient}(c, d, \cdot)$ 沿 $[a, b]$ 為仿射函數且兩端異號，其零點位於某個 $t \in (0, 1)$。 □

命題 25 的證明： 程式保留一個最佳候選，只在遇到鍵值嚴格較小的候選時才替換，因此在列舉的任何前段之後，它保存的是目前為止鍵值最小的第一個候選。近環結束後，若最小代價為 0，最小鍵值為 $(0, \ell_{\min})$，搜尋停止：結果是引線最短的第一個零代價候選。否則以同一個最佳候選繼續列舉遠環，結果是兩環中鍵值最小的第一個候選。每個步驟都是引數（極點、離開距離、開闊區域、交叉點）的函數，沒有隨機性，列舉順序固定。 □

A.5　樣式、主題與綁定

命題 27 的證明： 逐欄來看，合併是運算：$x \neq \mathrm{None}$ 時 $x \circ y = x$，否則為 y。$(x \circ y) \circ z$ 與 $x \circ (y \circ z)$ 都等於 x, y, z 中第一個不是 `None` 的值（或 `None`），`None` 是雙邊單位元，且 $x \circ x = x$。樣式在各欄相等時相等，由此得 (i)–(iii)，再由歸納法得到第一個非 `None` 值的描述。樣式組逐槽同理，`None` 槽的行為如同空樣式。 □

命題 28 的證明： 一對十六進位數字 $k \in \{0, \dots, 255\}$ 讀為 $k/255$，寫出時取 $\mathrm{round}(255 \cdot k/255)$ 的兩位大寫數字。在倍精度下，計算出的乘積與 k 相差至多 $255 \cdot 2^{-52} < 1/2$，因此捨入為 k。`include_alpha` 為真，即九個字元的形式時，才寫出 alpha 那一對。三位數形式在讀取前先把每位數字重複一次。 □

定理 30 的證明： 以 $\triangleright$ 表示合併。解析從 D 開始，依 T、G、C 的順序，對每個對應依序把 $M[k_n], \dots, M[k_1]$（由最一般到最具體）合併到結果之上，不存在的鍵給出空樣式組；最後合併圖層自帶的樣式組 E，它由 `style_slots` 指名的欄位構成。展開並以結合律（參見命題 27）去掉括號，解析後的樣式組為

$$E \triangleright C[k_1] \triangleright \cdots \triangleright C[k_n] \triangleright G[k_1] \triangleright \cdots \triangleright T[k_n] \triangleright D,$$

其各欄位為依此順序第一個不是 `None` 的值。之後綁定色盤只把顏色名稱換成顏色，不影響其他欄位。 □

推論 31 的證明： 角色為 `axes.note` 的文字圖層，其鏈為（`axes.note`, `axes`, `text`），在定理 30 的序列中 G[text] 位於 T[axes.note] 之前。依假設，更早的項目都沒有設定大小，因此第一個非 `None` 的大小來自 G[text]。 □

定理 33 的證明： 對 e 作歸納。若 $\mathrm{free}(e) \subseteq \mathrm{dom}\,\beta$（特別是常數，以及屬於 $\mathrm{dom}\,\beta$ 的參數），$\mathrm{bind}(e, \beta)$ 是一般值 $e(\beta)$：它沒有自由參數，符合 (i)，且求值為 $e(\beta) = e(\beta \cup \gamma)$，因為運算式的值只取決於其自由參數的值。若 e 是參數 $p \notin \mathrm{dom}\,\beta$，結果

為 p，$\text{free} = \{p\} = \text{free}(e) \setminus \text{dom}\ \beta$，值為 $\gamma(p) = (\beta \cup \gamma)(p)$。否則 $e = e_1 \circ e_2$，結果為 $\text{bind}(e_1, \beta) \circ \text{bind}(e_2, \beta)$（一般值包裝為常數）。其自由參數為

$$(\text{free}(e_1) \setminus \text{dom}\ \beta) \cup (\text{free}(e_2) \setminus \text{dom}\ \beta) = \text{free}(e) \setminus \text{dom}\ \beta$$

而 γ 綁定了兩部分各自的自由參數，由歸納假設其值為

$$e_1(\beta \cup \gamma) \circ e_2(\beta \cup \gamma) = e(\beta \cup \gamma)$$

□

推論 34 的證明：兩次應用定理 33 (i)，兩者的自由參數都是 $\text{free}(e) \setminus \text{dom}\ (\beta_1 \cup \beta_2)$。對這些參數的綁定 γ（與 β_1、β_2 不相交），兩次應用 (ii) 得

$$\text{bind}(\text{bind}(e, \beta_1), \beta_2)(\gamma) = \text{bind}(e, \beta_1)(\beta_2 \cup \gamma) = e(\beta_1 \cup \beta_2 \cup \gamma) = \text{bind}(e, \beta_1 \cup \beta_2)(\gamma)$$

畫布以這種方式綁定其場景中的每個運算式。 □

命題 35 的證明：由 $r = \lceil n/c \rceil$ 得 $rc \geq n > (r-1)c$，因此前 $r-1$ 列是滿的，最後一列有 $n - (r-1)c \in [1, c]$ 格，空格數 $rc - n < c$。由 $c = \lceil \sqrt{n} \rceil$ 得 $n \leq c^2$，所以 $n/c \leq c$，$r \leq c$。 □

更新紀錄

本章列出影響套件使用的版本變更，不列僅修改文件的版本。依 l3doc 慣例，每個條目都標註在它所描述的功能旁，因此所列頁碼就是該功能的實際頁碼。

v1.0.0rc6 (2026-10-23)
一般：各套件宣告 `econ_toml.Schema`，由 mosaickit 解析 `econ-viz.toml`；新增共用主題（`mosaickit.themes`）與標籤微調（`Nudge`） · 2
`Nudge`：新增 ····· 16
`Schema`：新增；套件宣告自己的鍵，由 mosaickit 解析檔案 ····· 25
`get_tokens`：新增 ····· 26
v1.0.0rc5 (2026-10-23)
一般：新增 `mosaickit.econ_toml`，讀取每個 EconViz 套件共用的 `econ-viz.toml` 區段 ····· 2
v1.0.0rc4 (2026-10-23)
一般：已解析的字型可跨符號連結共用；設定字型後的標題基線與 Matplotlib 的標題一致 · 2
v1.0.0rc3 (2026-10-22)
一般：標題與夾有 `$...$` 數學式的文字改用設定的文字字型排版，中文不再顯示為方框 ····· 2
v1.0.0rc2 (2026-10-22)
一般：繪製更快，輸出不變：標籤配置、點的檢查與 Matplotlib 的 mathtext 解析改用陣列，並使用只在單次繪製或儲存期間有效的快取 ····· 2
v1.0.0rc1 (2026-10-21)
一般：移除已棄用的 `Animation` 與 `TextStyle.family`（詳見參數與樣式兩章） ····· 2
`TextStyle`：移除 `family`；改以 `font` 選擇字型，並用 `TextFonts` 換成自己的字型檔。欄位請以名稱傳入；含 `"family": null` 的場景文件仍可載入，指定字型家族名稱的則會被拒絕 ·20
`Animation`：移除 0.6.0 起棄用的 `Animation`、`Animation.sweep()` 與 Matplotlib 播放輔助函式；動畫改由 kinetickit 負責，`CanvasGrid.sweep()` 與 `canvas.bind()` 仍為每個值畫一格 · 29
v0.9.2 (2026-10-21)
`TikZ`：`font` 對應到 TeX 字型家族 ····· 21
v0.9.1 (2026-10-21)
`TextStyle`：新增 `font` 欄位；`family` 棄用 ·· 20
`TextFonts`：新增 `serif` 與 `mono`；預設為 DejaVu ····· 20
v0.9.0 (2026-10-21)
`TextStyle`：`weight` 不再棄用 ····· 20
`TextFonts`：新增；sans 接一份檔案清單 ····· 20
`Config`：新增 `Config.math_font` 與 `Config.fonts` ····· 24
v0.8.0 (2026-10-20)
`DashStyle`：`DashStyle` 與 `ArrowStyle` 接受簡寫，讓其他套件可直接當作自己的線型與箭頭列舉 ····· 19
v0.5.1 (2026-10-03)
`MarkerLayer`：繪圖區邊緣的標記完整畫出；中心在繪圖區外的標記略過 ····· 7
v0.5.0 (2026-10-03)
`ArrowLayer`：虛線、點線或點劃線套用在箭身；箭頭維持實線 ····· 7
`CanvasGrid`：新增 `links` ····· 28
v0.4.0 (2026-10-03)
`BraceLayer`：內側大括號的標籤在尖端外沒有空位時改以引線拉出（先前會重疊並發出警告） ····· 8
v0.3.2 (2026-10-03)
`PointLabelLayer`：只有點嚴格位於區域內部時，該區域才算是點自己的區域；位於區域邊上的點，標籤仍放在區域外 ····· 15
v0.3.1 (2026-10-03)
`PointLabelLayer`：點標籤可以位於包含其點的填色區域內 ····· 15
v0.3.0 (2026-10-03)
`AxisSpec`：座標軸標題改放在箭頭尖端之外，不再置中於尖端 ····· 8
`mosaickit`：座標軸外的文字依欄排列：標記、外側大括號（每條軌道一欄）、註記；標記與註記會沿軸分散，互不重疊 ····· 8
點標籤先於區域標籤配置，區域的引線標註因此會避開它們；標籤文字依旋轉角度量測 ···· 15

AxisNoteLayer：預設主題新增 axes.note 角色 ····· 8
Obstacles：Placement.leader 可以是 None ····· 17
fits_inside：新增 place_point_label 與 place_beside ····· 18
Palette：主題與樣式以色盤名稱指稱顏色，在建立繪製計畫時依 Config.palette 解析；未知名稱引發指明角色、欄位與色盤的 ConfigurationError ····· 22
Config：新增 Config.palette，預設為 DEFAULT_PALETTE ····· 24
Config.load：TOML 接受 [palette] 表，樣式顏色可以是色盤名稱，並在載入檔案時檢查 · 25
SaveOptions：新增 expand（預設 True） ··· 30

v0.2.0 (2026-10-03)

mosaickit：內建繪製器在第一次使用時依名稱載入；核心不再匯入 Matplotlib 後端 ····· 3
LayoutWarning：新增，在沒有任何引線標註位置能避開所有障礙物時發出 ····· 6
PathLayer：新增 clip（預設 True）；箭頭不再於路徑末端重畫實線，虛線箭頭因此保持虛線 ····· 7
TextLayer：anchor 接受四個角 top-left、top-right、bottom-left、bottom-right ····· 7
ArrowLayer：箭頭不再被座標軸裁切 ····· 7
AxisSpec：座標軸預設組改畫實心三角箭頭；軸線不受裁切，在繪圖區邊緣保持完整寬度 ·· 8
mosaickit.layout：新增：標籤配置背後的純幾何模組 ····· 11
DEFAULT_PALETTE：新增：灰階、white、blue、red 與 teal；預設主題的顏色取自此色盤，primary、secondary 與 accent 改為藍、紅、青綠 ····· 22
resolve：themes.resolve 接受 overrides，在主題之後依序套用 ····· 24
Layer：圖層宣告 style_slots；Matplotlib 繪製改由各型別的登錄表驅動 ····· 30

v0.1.1 (2026-10-01)

一般：PyPI 上的套件資訊連結首頁、儲存庫、問題追蹤、更新紀錄與發布說明 ····· 2

v0.1.0 (2026-10-01)

一般：首次發布：不可變的場景與圖層、稀疏樣式、具命名空間的主題、畫布、可跨格的網格、參數運算式、動畫、以工作為範圍快取的 Matplotlib 繪製，以及嚴格的 TOML 設定 ···· 2

指令索引

A

ArrowLayer ····· 7
ArrowPlacement ····· 20
ArrowStyle ····· 19
assign_lanes ····· 10
AxisMarkLayer ····· 8
AxisNoteLayer ····· 8
AxisSpec ····· 8

B

box_frame ····· 8
Brace ····· 14
brace_outline ····· 14
BraceLayer ····· 8
build_axes ····· 8

C

CacheKey ····· 30
Canvas ····· 5
CanvasGrid ····· 28
CanvasGrid.from_layout ····· 28
CanvasGrid.sweep ····· 28
CanvasSpec ····· 5
Color ····· 21
Config ····· 24
Config.from_dict ····· 24
Config.load ····· 24
Constant ····· 26
crosshair_axes ····· 8

D

DashStyle ····· 19
DEFAULT_PALETTE ····· 21
distance_to_boundary ····· 12

E

econ_toml.load ····· 25
econ_toml.use ····· 25
expand_roles ····· 24
Expression ····· 26

F

Fill ····· 20
FillLayer ····· 7
fits_inside ····· 18

G

get_tokens ····· 26
GridLink ····· 28
GroupLayer ····· 8
gutter_columns ····· 11

I

Interval ····· 5

L

Layer ····· 6
Layout ····· 28
LegendLayer ····· 7
LegendStyle ····· 21
list_shared_themes ····· 26

M

Marker ····· 20
MarkerLayer ····· 7
MathFont ····· 20

N

Nudge ····· 16
nudge_from ····· 16

O

Obstacles ····· 17

P

Palette ····· 21
Parameter ····· 26
PathLayer ····· 7
place_beside ····· 18
place_callout ····· 17
place_point_label ····· 18
Placement ····· 17

point_in_polygon ····· 13
PointLabelLayer ····· 15
polygon_edges ····· 12
polylabel ····· 14

Q

quadrant_axes ····· 8

R

ray_exit ····· 13
Rect ····· 11
rect_hits_segment ····· 12
rect_inside_polygon ····· 13
rect_overlaps_polygon ····· 13
RegionLabelLayer ····· 15
register_builder ····· 29
register_pass ····· 29
RenderCache ····· 30
Renderer ····· 29
RendererRegistry ····· 29
RolePack ····· 24

S

SaveOptions ····· 30
Scene ····· 6
Schema ····· 25
segments_intersect ····· 12
Span ····· 28
SpanBraceLayer ····· 9
SparseStyle ····· 19
spread ····· 10
Stroke ····· 19
StyleBundle ····· 22

T

TextFonts ····· 20
TextLayer ····· 7
TextStyle ····· 20
Theme ····· 22
ThemeRegistry ····· 24
TRANSPARENT ····· 21

U

use_config ····· 24

參考文獻

Agafonkin, V. (2016,). *A New Algorithm for Finding a Visual Center of a Polygon*. https://blog.mapbox.com/a-new-algorithm-for-finding-a-visual-center-of-a-polygon-7c77e6492fbc

Ayer, M., Brunk, H. D., Ewing, G. M., Reid, W. T., & Silverman, E. (1955). An Empirical Distribution Function for Sampling with Incomplete Information. *The Annals of Mathematical Statistics*, *26*(4), 641–647. https://doi.org/10.1214/aoms/1177728423

Barlow, R. E., Bartholomew, D. J., Bremner, J. M., & Brunk, H. D. (1972). *Statistical Inference under Order Restrictions*. Wiley.

Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

de Berg, M., Cheong, O., Kreveld, M. van, & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3rd ed.). Springer. https://doi.org/10.1007/978-3-540-77974-2

Haines, E. (1994). Point in Polygon Strategies. In P. S. Heckbert (Ed.), *Graphics Gems IV: Graphics Gems IV* (pp. 24–46). Academic Press.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55